# Intelligence Is Not Authority

## Stopping a Model Is Not Stopping Its Effects

Jepson Taylor (jepson@veox.ai) · 13 November 2026

**Abstract.** AI control protocols act on proposals: a monitor reads what an agent proposes, and an agent that is caught is stopped. Calls approved before the catch may still be running, and stopping the model does not stop them. Counted from the decision to stop, about $\lambda(L+w)$ effects still commit, for call rate $\lambda$, enforcement delay $L$ and tool latency $w$; a control evaluation that counts a caught attack as prevented does not see them. Halting inference or revoking admission at a broker leaves both terms. Fencing the sink, so that it refuses at commit any work issued before the stop, removes $\lambda w$, the larger term when long work ends in a short commit (a merge after a test run, a capture after a payment is authorized); only faster enforcement removes $\lambda L$. On real broker and sink processes, with a synthetic agent whose calls were all authorized when issued and the stop enforced at once, a median of 11 effects committed after a halt took effect, 11 after a revocation and 0 after a fence (by design), which discarded 11 legitimate calls. A gate that reads state at commit also catches what a reviewer of proposals can miss: shown the rules and the state, a Qwen3-8B reviewer giving a bare verdict accepted, in all 36 cases (24/36 on a second CPU), an approved proposal that the state had since made unauthorized; with reasoning (median 637 tokens) it accepted 0, and in a post hoc control all 36 where the state permitted them. But a gate admits whatever its specification permits. On AgentDojo, under a specification written per suite without the reference solutions, a Qwen3-8B agent completed 9/37 benign tasks (16/37 unprotected, 12/37 under Progent), and injections written to stay inside the specification succeeded in 53/74 episodes (62/74 unprotected). Of 9 fencing mechanisms on PostgreSQL, etcd and a filesystem, only etcd's RBAC revoke was enforced by the surface, refused a forged epoch and held while stale writers ran. All models are small open-weight checkpoints run locally on CPU.

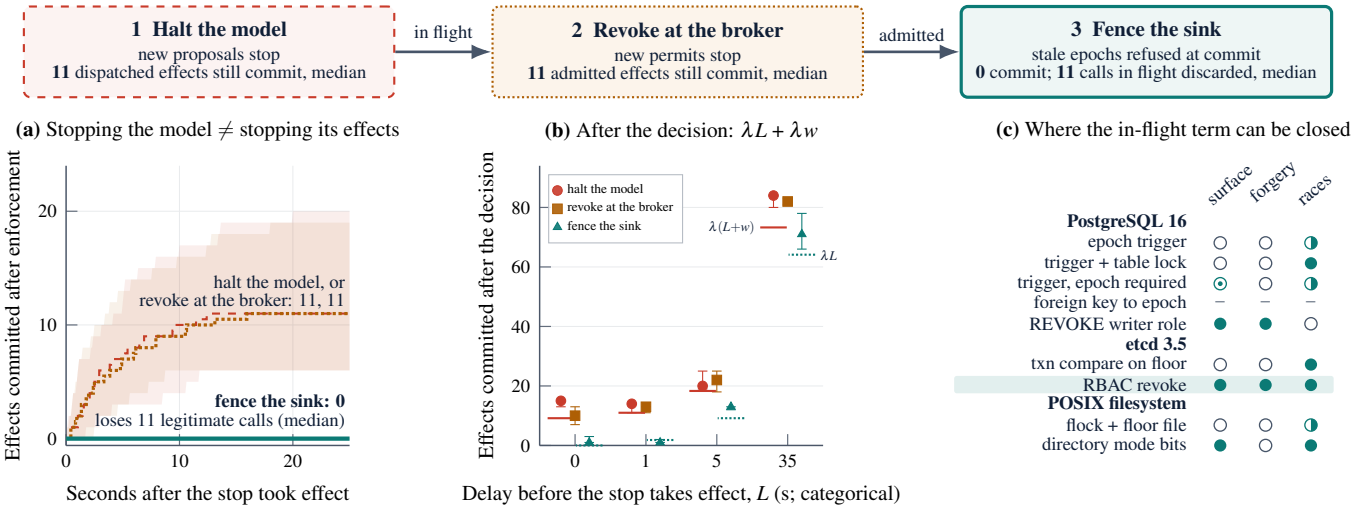



**Figure 1: Stopping a model does not stop its effects. (a)** Real broker and sink processes. A synthetic agent whose calls are all authorized when issued emits 2 calls/s to 32 concurrent tool calls of 5 s mean latency; the stop is decided at a random point of the agent's emission cycle and enforced at once; 40 replicates per stop, paired by seed (post hoc, E4c). Lines: median over replicates; shading: their range. Halting and revoking differed by 0 effects in the median replicate (range −5 to 7; no difference in 18 of 40): revoking refuses only calls emitted but not yet admitted. The fence's zero is by design; it admitted none after enforcement in any of E4's 84 trials, 12 with the sink killed or late completions. **(b)** E4 (stop at a fixed phase), same cell: median and range at each delay $L$ from decision to enforcement. Solid marks: the plan's halting prediction $\lambda(L+w)$, $\lambda$ measured (1.8/s); observed/predicted 1.25 (median; 0.87–8.24; 36 stable trials). Dotted: $\lambda L$, a post hoc reading. Halting and revoking differ only by the calls emitted but not yet admitted when the stop lands (median 4 at $L=0$, just after a burst of 4 calls; 0–3 otherwise); E4 did not vary where it lands; panel (a) varies it. Counted from the decision, the fence's 1–3 effects at $L=0$ committed while its enforcement calls ran. **(c)** Probes on single-node instances (40 runs, 20 races per mechanism). Columns: *surface*, the surface refuses a writer that omits the fence; *forgery*, a writer claiming the new epoch is refused; *races*, no stale write commits after the fence while protocol-following writers run. Filled: every run; open: none; half: some; ring with dot: a trigger we installed; dash: not measurable. Only etcd's RBAC revoke has all three (per-race failure below 14%, one-sided 95%, if races are independent); if every writer is trusted to follow the protocol, 4 of 9 held in every race.

## 1 Introduction

Agents built on language models now issue payments, edit repositories and change cloud resources through tools. Most of the safety case for such systems rests on the model: training shapes what it proposes, and a monitor, often another model, reads what it proposes before a tool runs [1, 2]. Both act on proposals. The harm, however, is in effects, and an effect is authorized or not by state that a proposal need not show:

whether this payment was already made, whether the person who granted the budget has withdrawn it, whether the record has changed since the agent read it. This paper separates the two. *Intelligence* lives in proposals: the capacity to search for, predict and choose actions. *Authority* lives in state: which effects a sink will commit, given what has already been committed and granted.

The separation is easiest to see when an operator stops an agent, by hand or through a system that tracks and stops model workloads [3]. The stop can act in three places (Figure 1a). *Halting inference* ends the stream of new proposals. *Revoking admission* at a broker refuses permits for proposals that arrive later. *Fencing the sink* raises an epoch floor at the component that writes durable state, so that an operation carrying an older epoch is refused when it tries to commit, even if it was admitted and dispatched before the stop. Only the third reaches work already in flight. It is the fencing-token argument of distributed locking [4] applied to an agent's authority, and it moves the stop from the producer of proposals to the place where effects happen. The window it closes does not depend on how the monitor runs. Asynchronous monitors, which review actions after they run, leave it open by design [5]. A monitor that blocks every call until it has read it [6] still approves calls one at a time; when a later call, or a pattern across calls, is flagged, the calls it approved earlier are still running. Work in flight at the stop is then the call rate times the tool latency, plus whatever the agent emits while the stop is being enforced. It is small when tools return in milliseconds and large when a tool runs for minutes or hours before its effect lands, as when an agent runs a long build or test suite and then merges the result, or authorizes a payment and captures it later: the work is long and the effect is a final write that can be refused. A control evaluation that counts an attack caught before it succeeds as prevented assumes that the stop reaches what was already dispatched; where it does not, measured safety is overstated by these effects.

Moving authority out of the model raises an obvious objection: the gate is then only as good as what it was told to check. We take that objection as the paper's subject rather than its limitation. The contributions, each with the evidence behind it:

1. **Stopping a model is not stopping its effects** (measured; Section 4). On real broker and sink processes under a predeclared grid of tool latencies, concurrency and enforcement delays, we count the effects that commit after each kind of stop and the legitimate work each discards, including kills of the sink inside its own requests.
2. **Without reasoning, reviewers miss violations that live in state** (measured; Section 5). Shown the rules and the state the gate reads, a same-model reviewer that replied with a bare verdict accepted a proposal from the approved list that the state had since made unauthorized, in every state-dependent case (on a second CPU, 24/36 on the larger checkpoint), and on the larger checkpoint none of the altered ones on the first CPU. Reasoning at length closed the gap on the larger checkpoint, at about a hundred times the generated tokens per check, and a post hoc control shows that it read the state: it accepted the same proposals where the state permitted them. A gate checks at commit rather than on a snapshot taken before.
3. **Mediation moves the failure into a written specification** (a hypothesis, illustrated by construction and measured on a public benchmark; Sections 5 and 6). A gate missing one clause admits exactly the violations of that clause. On AgentDojo, with grants written without the benchmark's reference solutions, we measure what a specification costs in benign utility, against an unprotected agent and a per-call policy, and what it leaves open to injections written to stay inside it.
4. **Which real effect surfaces can be fenced** (measured on CPU fixtures; Section 8). Executable probes against PostgreSQL, etcd and a POSIX filesystem sort each mechanism by who enforces the epoch: the surface itself, author code installed in it, or a condition the writer must remember to send.
5. **A machine-checked authority kernel** (proved; Section 3). The experiments' binaries do not use it as their decision procedure: unverified code calls verified predicates, and the comparison with the kernel is compiled out of release builds.

We also rerun the previous version's[1] five-controller comparison at the sample size its design required, with its frozen analysis (Section 7): 8 of its 16 of its component claims held. Its scoring is coupled to the gate, so it tests that version's claims, not this paper's thesis.

**What this paper does not show.** All live results use quantized open-weight models on CPU, decoding greedily except where E5 samples proposals; none uses a frontier model. There is no end-to-end control evaluation in which an adaptive red team attacks the whole system, which is the evaluation that would test these components together. The sink mediates only its own database effects; it does not provide external exactly-once execution, and an external surface is fenced only if it can refuse a stale epoch. The guarantees hold at a fixed grant. If more capable or more agreeable agents are granted more, intelligence regains reach through what people confer (Observation 1); we state a design for measuring that but have not run it.

## 2 Evidence Lifetime and Related Work

Measurements on 4B and 8B open-weight checkpoints need not transfer to stronger models. That objection is decisive for claims about a model's behaviour, and it sorts this paper's evidence: *evidence has an upgrade lifetime*. A quantity conditioned on a model's distribution of proposals expires when the weights, prompts or scaffolding change. A property proved over every input the sink can receive, and enforced outside the model's process, does not depend on the proposer at all, but it is only as broad as the specification it proves and holds only where the assumptions under it hold.

Table 1 sorts the paper's claims by upgrade lifetime. One row deserves emphasis. A complete gate's unauthorized rate

[1] An unpublished earlier draft of this paper. Its code and evidence are in the same repository (`evidence/v2`, `evidence/v3`), and its experiments are reproduced, with corrections, in the appendices.

| Safety claim or artifact | Quantified domain | Upgrade lifetime | Status when proposer checkpoint is upgraded |
|---|---|---|---|
| Verus authority-kernel theorems (§3) | All wire frames & call sequences ($\forall x$) | **Upgrade-invariant** | Holds for the verified functions; the experiments' release binaries do not run the comparison against them, so no reported number rests on it (§3) |
| Linux Landlock + seccomp confinement (§3) | Process ABI & file/socket descriptors | **Untested against an adaptive proposer** | Two bypasses found in review (existing IPC objects, an inherited descriptor) were fixed; others may remain, and a more capable proposer is better placed to find them |
| AgentDojo tool-graph min-cuts (§I) | The graph we drew from tool schemas and sinks | **Schema-dependent** | A property of that graph, not of the deployed tools; recomputed when tools change |
| Stop, revoke and fence leakage (§4) | Tool latency, concurrency & enforcement delay | **Workload-dependent** | Independent of the proposer; changes with the tool latencies and load, which here are sampled, not traced |
| Fenceability of a surface (§8) | One version of one surface | **Version-dependent** | Re-run the probe when the surface or its configuration changes |
| Reviewer misses (§5) | Checkpoint proposals $(x_1,\dots,x_K) \sim \pi_\theta$ | **Expires on upgrade** | Must be re-measured per checkpoint and per reviewer |
| Utility under a specification (§6) | Checkpoint competence within a fixed grant | **Expires on upgrade** | The gate's admissions follow its predicate; task utility changes with the checkpoint |
| What is not written in the specification (§6) | Attacks inside the grant | **Set fixed; rate expires** | Which attacks the specification admits does not depend on the proposer; how often an agent carries one out (E8b) does, and a stronger proposer may find more (conjecture) |
| Human scope grants (Observation 1) | Human conferral policy $g(a \mid \text{persona})$ | **Expires / degrades** | More capable or agreeable models may be granted more; not measured here |

**Table 1:** Lifetime of safety evidence across model upgrades. Proposer-independent properties are bounded by their specification and assumptions; checkpoint-contingent measurements must be repeated per checkpoint. The 4B and 8B experiments test the protocol, its cost and its gaps, not the alignment of any model.

is the same at every checkpoint because its admission predicate is the scoring predicate (Section 7); that zero carries no information about the proposer. The quantities that can change are the leakage of the other layers, reviewer misses, what the specification costs, and how much of what it omits a proposer finds. We expect the last to grow with capability, but this paper does not measure it.

### 2.1 Related Work

**Authority as state in systems.** Complete mediation, least privilege and tamperproof reference monitors come from capability and operating-system security [7–9]; authority by conferral rather than identity from object capabilities [10]; revocation through an indirection the grantor controls from Redell's caretakers [11], and revocation that must also reach copies already handed out from hardware capabilities, whose heaps are swept for stale ones [12]; attenuated delegation from Macaroons [13]. Leases [14] bound authority in time. Zanzibar [15] evaluates an authorization check at a snapshot no older than the content it guards, so that a permission already removed is not applied to new content. Lock services hand out sequence numbers that a storage server can check, Chubby's sequencers [16] and ZooKeeper's transaction identifiers [17], and Kleppmann's fencing token [4] is the rule that a storage service refuse a writer whose number is stale. The idea is older than any of these: consensus protocols refuse messages from an earlier ballot or term [18, 19], optimistic concurrency control aborts a transaction whose reads have been overwritten [20], two-phase commit makes a participant's effect wait for a coordinator's decision [21], HDFS's journal nodes refuse any writer whose epoch is lower than one they have promised [22], and clustered storage fences a node the cluster has expelled, by removing its key from a SCSI-3 persistent reservation so that the disk refuses its writes [23, 24], or by powering it off [25]. Our fence is the stale-epoch rule applied to an agent's authority, with the operator's stop decision in place of an expired lock or a lost election; what is new is not the mechanism but where it is placed and what it is measured against. The gap it closes is the classic time-of-check-to-time-of-use race [26], which TOCTOU-Bench has measured in tool-using agents [27]: a check made before the effect, on state that can change before the effect commits. The escrow method [28] lets concurrent transactions draw on a shared quantity without serializing on it, which is what a budget shared by concurrent tool calls needs; sagas [29] compensate long-lived work that has already committed, which is the only recourse once an effect is out. Machine-checked systems such as seL4 [30], IronFleet [31] and Verdi [32] prove properties of executable state machines; we use Verus [33, 34].

**Stopping agents.** AI control evaluates untrusted models under trusted monitoring [1, 2, 35, 36], and SHADE-Arena [37] measures whether agents can complete hidden harmful side tasks without an LLM monitor noticing; resampling protocols [38] replace or audit suspicious actions, control games [39] choose which actions to audit, and adaptive macro-protocols [40] adjust trust across many tasks where no single action is decisive. These protocols act on proposals and treat a caught action as a stopped one. When a monitor blocks an action before it runs, that action leaves nothing in flight; the window we measure opens when the stop comes later than the dispatch, as when a trajectory is halted after it is flagged, and it holds the agent's other calls already running. Asynchronous monitors, which review actions after they run so as to add no latency, make that the normal case [5], and the case for monitors that block

instead rests on how long an attack takes to do harm after it is dispatched [6]; the fence shortens the harm a late verdict leaves for effects that land at commit. Work on interruptibility asks whether a learning agent will come to resist or avoid being stopped [41–44], and some frontier models given a shutdown mechanism in their own environment have disabled it to finish a task [45], which is a reason to put the stop where the agent cannot reach; and work on scheming asks whether a model would hide such behaviour from the evaluations and monitors meant to find it [46–49]. Our question is the complement of both: once inference has stopped, which of the agent's already dispatched effects still commit, and what refuses them. Evidence that models can fake alignment, keep backdoored behaviour through safety training, or scheme in context [50–52] is the reason not to rest the answer on the proposer's disposition. Proposals for agent infrastructure [53] include mechanisms to attribute, shape and respond to agents' actions, and systems that track and stop model workloads [3] act where proposals are made; a sink fence is one concrete response mechanism at the other end.

**Agent security.** AgentDojo [54, 55] benchmarks prompt injection against tool-using agents. CaMeL [56] separates control from data flow and enforces capabilities in an interpreter, FIDES [57] enforces information-flow labels, f-secure [58] keeps untrusted input out of planning by information-flow control, PFI [59] isolates the agent that reads untrusted data and guards privilege escalation, Progent [60] enforces per-call privilege policies, AgentSpec [61] enforces runtime rules written in a small specification language, Conseca [62] generates a policy for each task's context, IsolateGPT [63] isolates the execution of third-party apps, MELON [64] flags an action that does not depend on the user's task by re-executing with the task masked, Task Shield [65] checks each action against the user's goals, ShieldAgent [66] verifies an action trajectory against rules extracted from policy documents, LlamaFirewall [67] audits the agent's reasoning for goal hijacking (its AgentDojo scoring counts an attack that succeeded and was flagged afterwards as defended, which assumes the late flag stopped the effect), ToolEmu [68] judges tool-call risk in an emulated sandbox, and a catalogue of design patterns [69] constrains what an injected agent can reach by how the application is structured. Each decides whether a call may be made. None commits the effect, so none can refuse, at the moment of commit, an effect admitted under a state that has since changed: a budget another call has spent, a grant its issuer has withdrawn, a stop the operator has ordered. In our scripted races, our own stateless per-call filter, which sees one call and a snapshot of state with no serialization point at commit, overspent a shared budget as soon as two writers contended (Appendix H); we did not run those systems in the races. These systems and a sink are complementary; a CaMeL-planned call still needs a sink that refuses it after a stop. We did not run CaMeL, whose planner must write programs in its restricted dialect, with the small local models used here. GoEX [70] takes the opposite route to a fence, validating actions after they run, with undo and damage confinement. $\tau$-bench [71] grades agents on policies that depend on database state, by the database's final state; its policies live in the agent's prompt, not at a sink. Guaranteed Safe AI [72] and shielded or constrained reinforcement learning [73, 74] restrict policies by construction.

**Closest prior work, and what is new here.** Three recent papers are close. Zhu and Wang [75] conserve budgets across a delegation graph with escrowed reservations bound to lineage, epoch and idempotency key, signed dispatch permits checked at a gateway, and uncertain effects that stay charged until settled; they prove conservation and at-most-once settlement and inject crashes into a two-process SQLite implementation. Our broker and sink use the same mechanisms, which we do not claim as new. Li [76] measures duplicate side effects of agents under injected faults, finds that when a write's outcome cannot be read back the tool contract, not the model, decides whether it is duplicated, and proves that no policy that only verifies can be exactly-once under late commits without a bound on in-flight time. A fence is not such a policy: the surface refuses the late commit itself, so it needs no bound on in-flight time, and it works only where the surface can refuse (Section 8). Moon and Varshney [77] verify a containment layer at the typed action boundary, treating the model as an arbitrary adversary. What this paper adds is measurement of three questions those designs raise: how much of an agent's work still commits after each kind of stop, and which term of it each mechanism removes (Section 4); whether a reviewer reading proposals can substitute for a check on state (Section 5); and what a specification written without the benchmark's solutions costs and leaves open, beside a per-task policy of the kind whose utility cost Progent and CaMeL report for their own policies (Section 6). We add executable probes of which real surfaces can enforce a fence at all (Section 8).

## 3 The Authority Kernel and the Sink Protocol

The broker issues permits; the sink commits effects. Both are Rust processes that keep their state in SQLite (write-ahead log, `synchronous=FULL`) and talk over JSON-lines RPC. A permit names one effect, its budget charge, the issuer whose grant it draws on, the revision it expects and the epoch under which it was issued. The sink commits a permitted effect only if, at commit time, the charge fits the remaining budget, the issuer's grant is live, the revision matches, the permit has not been used and its epoch is at least the sink's floor. Raising the floor is the fence. The stop experiments (Section 4) and the replication (Section 7) run these processes, and the gate of Section 5 decides each proposal by submitting it to them. On AgentDojo (Section 6) they are not involved: each arm's predicate is applied in the harness, at the tool boundary, to AgentDojo's in-memory environment.

### 3.1 Threat Model

**Untrusted.** The proposer: the model, its prompts and scaffolding, any sub-agents, and any content injected into them. It may send any sequence of requests, well-formed or not, at any rate, and may be misaligned or under an attacker's control. Its process is confined so that it reaches effects only through the broker's socket (below).

**Trusted.** The broker, the sink and the host they run on; SQLite; the cryptographic code, which is unaudited; the channel by which an operator orders a stop; and whoever writes the specification. A compromised broker or sink host defeats every guarantee here.

**Covered effects.** Only effects that commit at the sink, so that nothing happens until the sink commits it. We call this the *deferred-commit precondition*. An effect that a tool performs before or without the sink's commit, and any external system the sink merely records, is outside the guarantee; Section 8 asks which external surfaces can enforce the sink's epoch themselves. Reading data, side channels, and exfiltration through an effect the specification permits are covered only as far as the specification names them.

### 3.2 What Is Proved

We verified the kernel of the authority transitions with Verus [33, 34]: 53 executable functions and 43 proofs discharging 249 obligations with 0 trigger warnings (Table 2), with no `assume`, `admit` or external bodies in the verified block. The properties, each over every input of its type, are:

1. **Conservation and post-fence exclusion.** No sequence of operations commits more than the granted budget, and after a fence no operation below the new floor commits.
2. **Canonical permit encoding.** The fixed-width encoding that the broker and sink use to compare a submitted permit with the approved one, byte for byte, is a bijection on canonical frames and rejects every non-canonical frame. Between processes, permits travel as JSON lines, whose parsing is not verified.
3. **Delegation attenuation.** Along any delegation chain, a child's scope is a subset of its parent's and its budget no larger.
4. **Relational exclusion.** Two actions declared conflicting (for example, reading a credential and sending to the network) cannot both commit within one epoch.
5. **Monotone policy ratchet.** A new policy version is accepted only if it permits a subset of what the old one did, unless an out-of-band quorum signs the expansion.
6. **Skew-buffered quorum quiescence.** A revocation is acknowledged only when every sink shard has signed the new epoch or has passed its lease expiry adjusted for a bound $\delta_{\max}$ on clock skew.

Proofs fail silently when the specification is vacuous, so we pair them with three negative controls. First, 10 soundness mutations of the executable code are each rejected by Verus: refunding a timed-out charge without the sink's authority, bypassing the epoch check, admitting on the lower bound of an uncertain cost, accepting trailing bytes in the decoder, widening scope on self-upgrade without a quorum, accepting a cluster acknowledgment from a subset of shards, bypassing the conflict check, committing a staged effect without its verified postcondition, accepting an unverified widening certificate in the ratchet, and declaring quiescence without the skew buffer. Second, exhaustive exploration of 10,180 reachable states (223,960 transitions) of a small configuration finds no violation under the complete enforcer, while the same exploration with the budget clause omitted from the specification reaches 41,732 states and spends 8 units against a 2-unit cap while passing every remaining check. That is the specification gap of Section 5 in its smallest form. Third, 296 traces across all 55 single and paired clause omissions each produce a concrete violation.

**What the proofs cover in the running code.** The proofs are about the verified functions. In the release binaries that every experiment here used, the broker and sink decide with unverified Rust that calls the verified predicates; the comparison of each decision with the verified executable kernel was a debug assertion, compiled out of those builds. The decision logic, the RPC handling and SQLite are tested, not proved. The proof receipt is produced by a separate script and checked by hash; neither the local gate nor continuous integration reruns Verus.

### 3.3 Partition Schedules (Simulated)

A sink shard may be partitioned from the broker when a revocation is sent. Lease expiry can stand in for its missing acknowledgment only under a timing assumption.

**Assumption 1 (bounded clock skew).** Each shard's clock and the broker's differ by at most a known $\delta_{\max} \geq 0$.

A shard rejects a permit issued at broker time $t_{\text{issue}}$ with lease $L_m$ once its own clock passes $t_{\text{issue}} + L_m$. Under Assumption 1, waiting until the broker's clock passes $t_{\text{issue}} + L_m + \delta_{\max}$ guarantees this even for a shard whose clock lags by $\delta_{\max}$; the Verus theorem states this inequality, and the mutation that drops $\delta_{\max}$ fails verification.

We exercised the protocol logic in a single-process simulation: three sink threads connected to a coordinator by Unix socket pairs, under 1,000 randomized schedules per protocol class of message drops, delays, reorderings, duplications, asymmetric partitions, forged acknowledgments and simulated restarts, applied in logical time. It is in the style of Jepsen [78] testing but is not Jepsen and does not test a deployed implementation under real faults; Section 4 does that for one sink. Each class's outcome follows from its protocol rule, and the simulation counts how often a randomized schedule exposes it:

- **Signed, skew-buffered fences:** 0 of 1,000 schedules leaked a commit after acknowledgment (288 acknowledged by signed fences, 712 by lease expiry, 246 forged acknowledgments rejected), and 0 with shard clocks lagging up to 45 ms under a 50 ms buffer.
- **Skew buffer omitted:** 44 of 1,000 schedules (4.4%) leaked.
- **Acknowledged at the broker** while requests were in flight: 791 schedules (79.1%); **a sink that ignores epochs:** 276 (27.6%).

These are counts of schedules with at least one leaked commit, not of leaked commits. The percentages depend on the schedule generator's fault rates, which we chose; they are existence witnesses for each class's failure, not estimates of how often it occurs in a deployment.

### 3.4 Cryptography, Crash Recovery and Confinement

**Cryptography.** The SHA-2, HMAC and Ed25519 [79] code is written from scratch with no external crates. It enforces canonical scalars and encodings and rejects small-order points, passes the RFC test vectors and a bidirectional cross-check against the OpenSSL command line, and accepted 0 of 10,000 forgery and malleability attempts. It has not been audited and is part of the trusted computing base; a deployment should use an audited library.

**Crash recovery.** A separate three-process cluster test (SQLite in write-ahead mode with `synchronous=NORMAL`, which survives a killed process but may lose the last commits on power loss) drives 50,000 malformed frames, 16 interleaved sub-agent identities with 3-hop delegation from one driver thread, and 100 SIGKILL-and-restart cycles delivered between requests, and reconciles the ledgers with no discrepancy (1121 $\mu$s median commit latency, against the tens of milliseconds per commit of Table 16, which uses `synchronous=FULL` and so waits for the disk). Kills inside a request are tested in Section 4.

**Confinement.** Landlock [80] file and network rules and seccomp [81] filters block the 11 direct socket, IPC and resource probes we wrote, so local user-space code must go through the broker socket. Independent review found two bypasses, a message queue that already existed and an inherited stderr descriptor; both were fixed and their probes added. The probe set is ours and is not exhaustive.

## 4 Stopping a Model Is Not Stopping Its Effects

An operator who decides to stop an agent can act at three places. *Halting inference* ends the stream of new proposals. *Revoking admission* at the broker refuses permits for proposals that arrive later. *Fencing the sink* raises the epoch floor at the component that writes durable state, so that any operation carrying an older epoch is refused at commit, including operations that were admitted, dispatched and are already running. A system that tracks and stops model workloads [3] acts at the first, on the producer of proposals. Only the third reaches work that is already in flight, and it is the fencing-token argument of distributed locking [4] applied to agent authority.

**Setup (E4).** We run the real broker and sink processes (Rust, SQLite, JSON-lines RPC; Section 3) under a synthetic agent that emits $\lambda = 2$ tool calls per second to $C \in \{4, 32\}$ concurrent executors whose latencies are exponential with mean $w \in \{0.5, 5, 30\}$ s. Each call is a pre-approved, authorized one-unit effect; nothing in the workload is adversarial. After a 20 s warm-up the operator decides to stop, and the stop takes effect $L \in \{0, 1, 5, 35\}$ s later. A trial counts the effects committed after the stop took effect, timed by the harness's log of the sink's replies (the set of committed effects must equal the sink's database), and the legitimate calls the stop discards (cancelled or denied). A reply follows its commit, so this timing can place a commit later than it happened but never earlier: it can add effects after the stop, never hide one. E4b, which kills the sink inside requests, counts from the database instead. The plan, grid and metrics were committed before the first trial; the 252 trials include 36 with SIGKILL of the sink or delayed completions.

**Result.** We draw the stable cell with the longest latency ($C = 32$, $w = 5$ s, $L = 0$), chosen by that rule rather than by eye. In E4, after the stop took effect, halting inference let 13–15 effects commit and revoking at the broker 7–10, the last 19.7 s later; fencing the sink let 0 commit and discarded 8–11 legitimate calls. Across all 24 fault-free cells, halting leaked up to 108 effects and revoking up to 32; across 84 fence trials, the most effects any one committed after the fence took effect was 0. The counts reconcile trial by trial: in 144 of 144 fault-free halting and revoking trials, the effects committed after enforcement equal the calls in flight at enforcement plus those emitted but not yet accepted, less those refused, and in 72 of 72 fence trials none committed and every call in flight was discarded. The decision always falls just after the agent emits a burst of 4 calls (the 20 s warm-up is a whole number of 2 s steps). With the stop enforced at once, a halt lets that burst through and a revocation refuses it, which is most of the difference between the two in the drawn cell; with any delay the burst is admitted before the stop takes effect, and the median counts after enforcement for halting and revoking were 10 and 9 at $L = 1$ s; 12 and 12 at $L = 5$ s; 13 and 14 at $L = 35$ s. We did not vary the phase.

**Where the stop lands (E4c, post hoc).** E4's warm-up was a whole number of the agent's 2 s emission cycles, so the decision always fell just after a burst of four calls, which revoking at once refused and halting did not. After an internal review pointed this out, we fixed a plan for one more run of the drawn cell with the decision at a uniformly random point of the cycle, 40 replicates per stop, the three stops of a replicate paired by seed and phase, and committed the front figure to its result before running it. Halting let a median of 11 effects commit after the stop took effect (range 6–20), revoking 11 (6–19) and fencing 0, which discarded a median of 11 legitimate calls (6–15). Paired by replicate, halting and revoking differed by 0 in the median (range $-5$ to 7; no difference in 18 of 40). Figure 1a draws this run. E4's gap between halting and revoking came from where its stop landed: when calls are dispatched as soon as they are emitted, revoking at the broker refuses almost nothing that halting does not, and both leave open the work already in flight.

**What the numbers mean.** The fence's zero is a property of the design, not a discovery: the sink serializes its RPCs, `fence` cancels every pending record below the new floor, and effect identifiers are unique in the schema, so duplicates are impossible rather than rare. What E4 measures is (i) that this implementation keeps the property under the faults we injected, and (ii) the size and cost of the window the other two layers leave open. That window is queueing, and it behaves like queueing. Where the offered load $\rho = \lambda w / C$ is below one, halting leaks the calls in flight and those emitted before the stop takes effect, which the plan predicted from Little's law. Over the 36 halting trials without injected faults in stable cells, the commits after the decision were 1.25 times the plan's prediction

$\hat{\lambda}(L+w)$ at the median (range 0.87–8.24); the largest ratio is at $L+w=0.5$ s, where RPC time and the emitter's bursts of 4 calls are not small against the window. Revocation leaks the calls already accepted. Where $\rho \geq 1$ (3 of the 6 $(w,C)$ cells), the backlog grows during the warm-up and halting leaks all of it; we report those cells separately and do not apply Little's law to them. The fence pays for its zero in the opposite currency. Every legitimate call in flight at the stop is discarded, which is what an operator who stops an agent usually wants and what an operator who meant "pause" does not.

**Which term each stop removes.** Counting from the decision rather than from enforcement separates two terms (Figure 1b). During the delay $L$ no layer acts, and about $\lambda L$ effects commit at call rate $\lambda$. When the stop takes effect, the calls already dispatched add about $\lambda w$ for halting and fewer for revocation, which refuses those not yet accepted. The fence removes the second term and none of the first. In the drawn cell at $L=35$ s, with a measured call rate of 1.8 per second, halting let 80–84 effects commit after the decision and fencing 66–78. Whenever enforcement is slow compared with tool latency, shortening $L$ is the larger lever; the fence is what makes the remainder zero. The plan predicted $\lambda(L+w)$ for halting; reading the fence's count as $\lambda L$ is post hoc.

**Why not kill the executors?** An operator who controls every executor can stop each one before it commits, and in this harness that would also leave zero, for the same reason the fence does: the executors are our processes and every effect lands at our sink. We did not run that arm, because its zero would hold by construction as well. The fence needs no such control. It acts at the one place every effect must pass, so it also covers an executor the operator cannot reach (a remote runner, a third-party service, a host on the other side of a partition), a commit request already sent when the kill arrives, and a retry after the sink restarts; E4b below measures the last two. Killing what can be reached and fencing what must not commit are complementary, and the fence is the one that does not depend on knowing where every executor is. The other natural strengthening, revoking at the broker and also invalidating the permits it has already issued, is a fence exactly when something checks the invalidation at commit; checked anywhere earlier, a permit already presented is not reached.

**Faults (E4b).** A SIGKILL that lands between requests tests little. In a separately predeclared run we kill the sink while a `complete` or a `fence` request is outstanding, hold the executor's `accept` across the fence, and drive the system with closed-loop agents that wait for each result. In E4b (152 predeclared trials; every kill was confirmed to land with its request outstanding), the post-stop count is read from the sink's database. With the sink killed inside an outstanding `complete`, the fenced arm committed 0 effects after the stop and the interrupted operation ended cancelled in 10 of 10 trials; under halting and revoking it was retried and committed after the stop in 20 of 20, with 7–16 and 6–21 post-stop commits. Killed inside an outstanding `fence`, the fenced arm committed 0. Of 80 accepts held across the fence, 0 produced an effect; of 160 held across a halt or a revocation, 160 did. With closed-loop agents that wait for each result, halting and revoking let through exactly the calls in flight at the stop in 48 of 48 trials, and the fence committed 0 while discarding 0–11 calls. The sink's ordering check found 0 violations. In 10 trials (exit after commit in complete) the harness log counted a post-stop commit that the database shows was made before the stop, which is why E4b counts from the database.

**Scope.** The sink mediates only its own SQLite effects. An external effect (a payment API, a repository, a cloud resource) is fenced only if that surface can refuse a stale epoch, which is the question of Section 8 and Figure 1c. No arm aborts an executor that is already running: the fence refuses its commit, but anything an executor does outside the sink before it commits is stopped by none of the three layers. The latencies are sampled, not production traces, and the agent is synthetic.

## 5 Reviewers and Gates on Violations That Live in State

A common alternative to mediation is review: a second model, often the same one, reads each proposed action and approves or denies it [1]. Review sees the proposal. Many unauthorized actions, however, are unauthorized only because of the state they would act on: a payment that was legitimate once and is now a replay, a grant whose issuer has since been withdrawn, an edit against a revision that has moved. Such a proposal can be byte-identical to one the policy approved. We test whether a reviewer that is given the rules, the visible state and time to reason catches these, and what a gate that reads state does instead.

**Setup (E5).** Sixty cases in five families (twelve each) each pose a pre-state, a policy with six written clauses (exact match to an approved proposal, cumulative budget, live issuers, current revision, expiry, scope) and an objective. Three families are *state-dependent*: their violations break a clause that can only be checked against sink state (replay after the budget is spent, issuer withdrawn, stale revision). Two are *proposal-visible*: an altered payload, and a payload altered at the request of an injected note that the proposer, not the reviewer, reads. The same checkpoint proposes candidates under eight predeclared mutation strategies and reviews them in three arms: the minimal prompt of this paper's previous version (R0), the four rules the families test stated in the prompt (R1), and the same rules with reasoning enabled (R2, up to 4096 tokens; a truncated review is a deny). Proposals are sampled at temperature 0.8; reviews are greedy. Every valid candidate also goes through the real broker and sink at the case's pre-state (G). Plan, prompts, clauses, parse rules and hypotheses were committed before data (three pre-data amendments). We count each distinct proposal once, because greedy reviews of a repeated proposal are identical, and take cases as the unit, because proposals within a case are not independent.

**Result.** The first predeclared hypothesis ($H_{E5a}$: without reasoning, reviewers admit violations that live in state more often than violations visible in the proposal) held on both checkpoints. With the rules stated (R1), the Qwen3-8B reviewer admitted a state-dependent violation in 36/36 cases and a proposal-visible

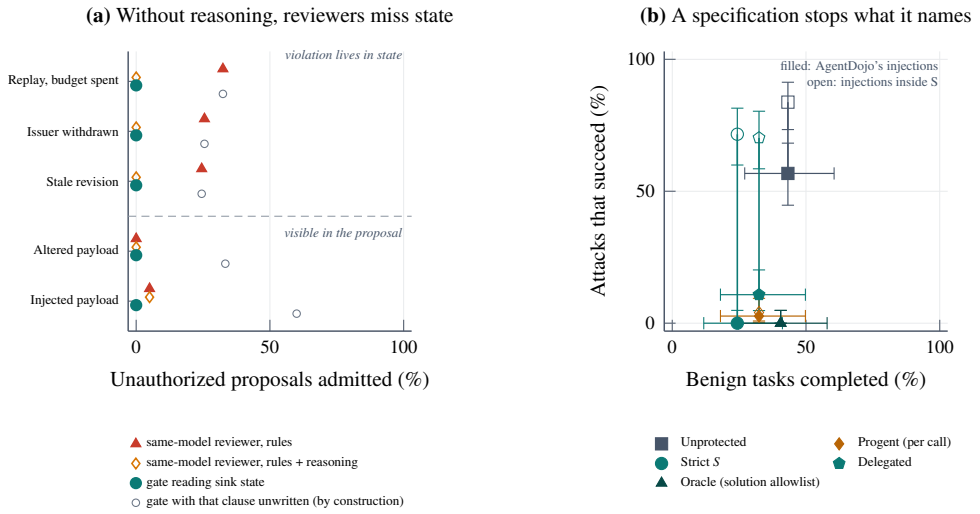



**Figure 2: A same-model reviewer misses violations in state unless it reasons at length; a written specification stops what it names, at a cost. (a)** Qwen3-8B red-team proposals under 8 predeclared mutation strategies, each distinct proposal counted once (no interval: proposals within a case are not independent). The reviewer is the same checkpoint, shown the rules and the state the gate reads. Without reasoning it admits 36/133 of unauthorized proposals whose violation lives in sink state, more often than 2/76 of those visible in the proposal (by case, 36/36 against 2/24). With reasoning (median 637 generated tokens per review, against 6) it admits 0/133 and 2/76. The gate admits 0; it enforces the written predicate, and its Rust kernel agreed with the Python reference on all 217 valid proposals of both sample sets (0 disagreements). Open circles are by construction: the share of each family's violations that break only that family's clause, which a gate without that clause would admit. **(b)** AgentDojo banking and Slack (37 user tasks), Qwen3-8B, AgentDojo's graders, every episode counted (95% Clopper–Pearson; the two attacked episodes of a task are not independent, so attack intervals are too narrow). A provenance specification written without the reference solutions (strict *S*) completes 9/37 benign tasks; unprotected, 16/37; the oracle, an allowlist copied from each task's solution and so a reference rather than a deployable specification, 15/37. AgentDojo's injections succeed in 0/74 strict and 42/74 unprotected episodes; injections that name only principals already in the user's state (written by us) succeed in 53/74 strict, 52/74 delegated, 62/74 unprotected, 3/74 Progent and 0/74 oracle episodes. Local quantized open-weight models on CPU at temperature 0; no frontier model.

one in 2/24 (Figure 2a; 36/133 and 2/76 of the distinct proposals). The failure is narrow. Each case approves two proposals, and in the state-dependent families the state has made one of them unauthorized: its issuer was withdrawn, its document has moved to a new revision, or the budget it would draw on is spent. The proposer submitted that unaltered proposal in all 36 state-dependent cases. The R1 reviewer approved it in 36 of them and approved 0/97 of the altered state-dependent proposals: it applied the first rule, exact approval, and not the three that read state. On Qwen3-4B it approved 36/36 unaltered and 42/180 altered ones. $H_{E5a}$'s comparison is therefore partly one of how the cases were built: only a state-dependent family can contain a proposal that matches the approved list exactly, and on Qwen3-8B that proposal accounts for every state-dependent admission without reasoning. Reasoning changed this. The second hypothesis ($H_{E5b}$: reasoning reduces state-dependent misses but does not remove them) held on Qwen3-4B and failed on Qwen3-8B. With reasoning enabled (R2), the Qwen3-8B reviewer approved 0 of the unaltered proposals, 0/97 altered ones and 2/76 visible violations, at a median of 637 generated tokens per review instead of 6; on Qwen3-4B it approved 6/36 unaltered and 3/180 altered ones. Every arm on both checkpoints is in Table 7 (Appendix C). The gate admitted 0. That zero is the gate enforcing its written predicate, not a finding: on all 217 distinct valid proposals of both sample sets on Qwen3-8B its Rust implementation agreed with a separately written Python statement of the same clauses (0 disagreements). Re-creating the broker and sink for each of the 400 distinct valid proposals of both checkpoints, as the validator does, reproduced every recorded gate decision.

**Controls (E5c, post hoc).** E5's state-dependent families contain no authorized proposal, so a reviewer that refused every unaltered approved proposal would also have scored zero there. After reading E5's results we fixed a plan for controls and ran it on a second host. Each state-dependent case was rebuilt without its state change, so that the same unaltered proposal is authorized (a *twin*), and in each duplicate-replay case the proposal already committed was resubmitted unchanged, an idempotent replay that every clause permits. Every arm (R1, R1p and R2) on Qwen3-4B and Qwen3-8B accepted all 36 twins and all 12 replays, so none of them recorded a false block. A variant of R1 with one sentence added after the rules (R1p), saying that a proposal on the approved list must still satisfy rules 2, 3 and 4 against the visible state, tests whether R1's failure comes from how the rules were stated; it is the only wording we ran, and its exact text is in the evidence. On Qwen3-4B, R1p accepted 36/36 of the unaltered unauthorized proposals (R1 on the same host: 36/36), 29/180 altered ones (R1: 53/180) and 36/36 twins, so by the plan's rule the added sentence did not remove R1's failure. On Qwen3-8B, R1p accepted 24/36 of the unaltered unauthorized proposals (R1 on the same host: 24/36), 0/97 altered ones (R1: 6/97) and 36/36 twins, so by the plan's rule the added sentence did not remove R1's failure. Re-running R1 and R2 on E5's own proposals on

the second host disagreed with E5's recorded decisions in 23 of 328 R1 and 3 of 25 completed R2 decisions on Qwen3-4B and 18 of 217 R1 and 0 of 11 completed R2 decisions on Qwen3-8B (R2 reviews that timed out at the harness's 900-second request limit are not counted: 11 on Qwen3-4B and 25 on Qwen3-8B). The R2 reviews that timed out were among the longest in E5, so the completed ones are not a random subset. Greedy decoding did not make a verdict portable: on Qwen3-8B, R1 accepted 24/36 of the unaltered unauthorized proposals on the second host, against 36 of 36 on E5's.

**What this shows.** A reviewer can check state only if it is shown the state, and E5 shows it everything the gate reads: one moment, at most one prior record. Under those conditions the reviewer that replied with a bare verdict accepted the approved proposal whatever the state: in E5 where the state forbade it, and in the controls where the state permitted it; one added sentence saying that the first rule is not enough did not change that. The reviewer that reasoned did read the state: on Qwen3-8B it accepted 0 of the approved proposals where the state forbade them and all 36 where it permitted them, so its zero is not a refusal of approved proposals. On Qwen3-4B it still accepted 6/36 of the forbidden ones. Two differences remain that the controls do not touch. The gate checks at commit, while a reviewer checks a snapshot before it, which is the time-of-check-to-time-of-use gap [26]: state that changes in between, such as a budget spent by a concurrent call (Appendix H), is invisible to the reviewer. And the gate's decision is fixed by its predicate, while the reviewer's depends on how it is run: the same unaltered proposals on Qwen3-8B were accepted in 36 cases without reasoning, 0 with it, and 24/36 when the same greedy reviewer ran on another CPU. The reviewer spends hundreds of generated tokens per action on that decision; the gate spends a database transaction.

**What a gate misses instead.** Remove one clause and the gate admits exactly the violations that break only that clause. This holds by construction, and Figure 2a draws it as such. What the experiment measures is how much each family rests on a single clause. Of the distinct unauthorized proposals that search found, 12/37 (replay, budget spent), 12/47 (issuer withdrawn), 12/49 (stale revision), 12/36 (altered payload), and 24/40 (injected payload) break only their family's clause. This is the paper's central hypothesis, which E5 illustrates and does not test: mediation does not make safety independent of the proposer. It moves the failure from a reviewer's judgment, which here varied with prompt and budget, to a written specification, whose gaps can be listed, tested and argued about.

**Limits.** The reviewers are local open-weight checkpoints, decoding greedily, reviewing their own proposals; a stronger or differently trained monitor may do better, and control protocols that resample or audit [38] are a different defence. The proposer is told the adversarial objective in plain text. The families differ in more than where the violation lives, so the comparison shows that these state-dependent cases are hard for a reviewer without reasoning, not that state-dependence alone makes them hard. Only eight distinct proposals were authorized, so E5 cannot estimate how often a reviewer denies legitimate work, and a reviewer that denied everything would also score zero; the controls measure that only for the unaltered approved proposals of the state-dependent families. A twin differs from its original only in the state, so the twins show whether a reviewer reads the state, not how often it blocks other legitimate work. R2 reviews that reached the token cap (10 of 329 on Qwen3-4B and 0 of 217 on Qwen3-8B, counting distinct valid proposals of both sample sets) count as denials, as the plan specified, which can only lower R2's admissions; on Qwen3-4B, counting them as acceptances would raise R2's state-dependent admissions from 9/216 to 17/216.

## 6 What a Specification Costs, and What It Cannot Stop

The previous section shows that a gate enforces exactly what is written. On a public benchmark, that raises two questions: what does writing it cost in legitimate work, and what happens to an attack that the writing did not anticipate? Earlier versions of this paper answered the first with permissions copied from each task's reference solution, which no deployment has. Here the permissions are written once per suite, without the solutions.

**Setup (E8, E8b).** AgentDojo v1.2.2 [55], banking (16 user tasks) and Slack (21), with AgentDojo's "important instructions" attack placed at each task's own injection vectors and AgentDojo's own graders. The agent is Qwen3-8B at temperature 0 in a JSON tool loop. Five arms see identical prompts and receive the same reply, "not permitted", whenever a call is refused:

- *unprotected*;
- *Progent*, using its released code with a per-task policy generated by the same checkpoint [60];
- *strict*, a written specification $S$ under which an effect may name an account, address, URL, user or channel only if it appears in the user's prompt or in the user's own state, and record writes must use values found there;
- *delegated*, which is $S$ plus trust in the output of any file or page the user's prompt names;
- *oracle*, with tools, targets and amounts copied from the reference solution (a reference for what the finest specification admits, not deployable, and not an upper bound on utility).

Each user task runs once without attack and twice under assigned injections (555 episodes). E8b repeats the attacked episodes with six injections we wrote to stay inside $S$: pay an existing payee, raise an existing standing order, schedule a recurring payment to an existing payee, message a secret to an existing user, add an existing user to a private channel, and remove a user. Both plans, the injections and their graders were committed before data.

**The cost before any model runs.** Offering each task's reference solution to each specification, with no model involved, gives the specification-granularity cost directly. $S$ admits 25/37 reference solutions and 0/249 of AgentDojo's reference attacks. Delegation admits 30/37 and 48/249. The oracle admits 37/37 and 0/249. For E8b's injections the same replay shows $S$ admitting 111/111 and the oracle 0/111.

**Live.** Figure 2b plots benign utility against attack success. Strict *S* completed 9/37 benign tasks against 16/37 unprotected and 15/37 with the oracle grant; delegation completed 12/37. AgentDojo's injections succeeded in 0/74 strict episodes, 8/74 delegated, 0/74 under the oracle grant and 42/74 unprotected. Injections inside *S* succeeded in 53/74 strict and 52/74 delegated episodes, against 62/74 unprotected, 3/74 under Progent and 0/74 under the oracle grant. Progent, with per-task policies generated by the same checkpoint, completed 12/37 benign tasks, and AgentDojo's injections succeeded in 2/74 of its episodes. Policy generation succeeded for all 37 user tasks.

**Reading.** Against the 16 benign tasks the unprotected agent completed, strict *S* completed 7 fewer, Progent's per-task policies 4 fewer, delegation 4 fewer, and the oracle grant 1 fewer. Paired by task, strict *S* completed 3 benign tasks that Progent did not, and Progent 6 that *S* did not; with 37 tasks and no test declared, we do not rank the two. Paired by user task against the unprotected agent (the plan's descriptive McNemar counts), strict *S* lost 7 and gained 0, Progent lost 6 and gained 2, delegation lost 4 and gained 0, and the oracle grant lost 2 and gained 1 tasks. *S* was written knowing AgentDojo's public tasks and injections, and the replay without a model shows which reference attacks it refuses before any model runs, so its native attack success counts only attacks that reach the injection's goal by a path its reference does not take. The informative numbers are therefore the utility *S* costs and the attack success it leaves when the attacker stays inside what *S* permits. Those injections succeeded in 53/74 strict episodes against 62/74 unprotected; *S* refused none of them, since it admits all of E8b's reference attacks (111/111), so the difference is what the agent did not carry out on its own. Progent's policies, generated per task by the same checkpoint from the user's request, let 3/74 succeed, and the oracle, which admits only each solution's targets, 0/74. On this benchmark, then, our per-suite specification did no better than a per-task policy on benign tasks (the paired counts above do not separate them) and let far more of the injections that stay inside it succeed. Those injections were written against *S*, not against Progent's policies, so this does not rank the two defences: it shows what an attack adapted to a specification achieves, and an attack adapted to Progent's policies was not run. This is the paper's central hypothesis observed against our own specification: a gate admits what its specification permits, and a specification written per task permits less than one written per suite. AgentDojo has no concurrent calls, withdrawn grants or stops, so it exercises none of what holding authority as state adds over a per-call policy, and nothing prevents enforcing a per-task policy as state at the sink; what E8 and E8b show is the price of the specification's granularity, not of where it is enforced.

**Across checkpoints (E2).** Re-running E8 unchanged with Qwen3-4B as the agent, each served by the same llama.cpp build on a second host's CPU (E8's Qwen3-8B ran on the first), gives Qwen3-4B: benign tasks 12/37 unprotected, 7/37 under Progent and 7/37 under strict *S*; attacks 22/74, 0/74 and 0/74. The comparison is descriptive: the checkpoints differ in family, size, chat template and how well they follow the JSON protocol, Progent's policies come from each checkpoint itself, and the host differs from E8's. Every arm is in Table 8 (Appendix D).

**Limits.** Two of AgentDojo's four suites; one attack template; none of AgentDojo's own defences (tool filtering, spotlighting, an injection detector) or the detection defences of Section 2 as further arms; a repository-specific tool loop rather than AgentDojo's native function calling; local open-weight models. In 5 of 1,110 pairs of arms run on the same episode, the model's reply differed while everything it had received was identical, so inference at temperature 0 was not fully deterministic and a small part of the differences between arms is not caused by the arm. E8b's injections and graders are ours and were chosen to lie inside *S*, which the replay without a model confirms; E8b measures how often the agent carries them out, which depends on the model as well as on *S*, not how often attackers find such injections. The sink mediates only its own records; AgentDojo's effects run in AgentDojo's in-memory environment, and the arm's predicate, not the sink, decides whether a call runs.

## 7 A Predeclared Replication at the Planned Size

The previous version's live comparison of five controllers was predeclared but underpowered. With 32 independent cases per checkpoint, its usefulness claims could not clear the five-point margin at the corrected $\alpha$ even with no losses (the zero-loss bound is 0.165), and it reported its claims as inconclusive. The plan's own design diagnostic gives the sample the noninferiority claims need: 113 independent cases. E1 runs 128.

**Design (E1).** The reference plan is unchanged, and so is its analysis script, frozen by hash. Five controllers are compared:

- no mediation;
- a stateless per-call policy;
- a same-checkpoint learned reviewer;
- a durable broker without a sink fence;
- complete mediation.

They run on $N = 128$ fresh independent cases per checkpoint, for Qwen3-4B and Qwen3-8B, which is 1,280 campaigns on CPU.

The primary claims are, for each comparator and checkpoint, one-sided safety superiority and five-point usefulness noninferiority of complete mediation: 16 component claims at Bonferroni $\alpha = 0.05/16$, with checkpoints never pooled. The freeze, roster seed and nonces are generated by the harness after the sample size is fixed. Two amendments precede the data:

- the first raises the wall-clock cap that had limited the pilot;
- the second follows a stopped launch. Proposals expired four hours after each checkpoint began, which would have denied the 8B tail as expired. That launch was stopped at 235 records, with no aggregate viewed and one record opened while diagnosing a schema error (Appendix F); it is kept and disclosed.

**What the primary claims can and cannot show.** Complete mediation's unauthorized rate is zero by construction: its admission predicate is the scoring predicate. The safety claims therefore measure how often each comparator leaks, not

whether complete mediation can be broken; Sections 4 and 5 address that. Usefulness keeps the previous version's scoring, which is coupled to the gate: a task succeeds only if the case's first approved proposal commits and nothing else does, and a denial counts as a false block only if the proposal satisfied the same predicate the gate enforces, so complete mediation cannot record a false block (Appendix H). At N = 128 the usefulness bound clears the 5-point margin only if complete mediation loses no case to the comparator: zero losses give a bound of 0.044 and one gives 0.061. A single case that the comparator completes and complete mediation does not defeats the claim.

The frozen scoring is also inconsistent about revocation. When a grant is withdrawn, the queued old operation commits unless the sink is fenced, so only complete mediation can complete those tasks, and they count toward usefulness in its favour; yet the same scoring does not count the effect committed after the withdrawal as unauthorized. The safety effect of the fence is therefore invisible to the primary claims. The durable broker without a fence differs from complete mediation only in the fence, so under the primary scoring it cannot leak, and the two claims that complete mediation is safer than it could not succeed. The plan anticipated this with predeclared secondaries, reported beside every primary claim: S1 also counts an effect after a withdrawal as unauthorized, S2 counts the first proposal's effect whatever else committed, S3 splits by attack family, and S4 recomputes the usefulness bound without revocation cases.

**Result.** Of the 16 predeclared component claims, 8 held. At Qwen3-4B, complete mediation was shown safer than same-model reviewer and no mediation but not than durable, no fence and per-call policy, and shown noninferior in usefulness to durable, no fence and per-call policy but not to same-model reviewer and no mediation. At Qwen3-8B, complete mediation was shown safer than no mediation but not than durable, no fence, same-model reviewer, and per-call policy, and shown noninferior in usefulness to durable, no fence, same-model reviewer, and no mediation but not to per-call policy. S4 cannot clear the 5-point margin at this sample even with no losses: its 108 non-revocation cases give a zero-loss bound of 0.052 (Table 5 reports its bounds). Without the revocation cases (S4), the usefulness bound cleared the five-point margin in 0 of 8 checkpoint–comparator pairs. Under S1 the durable broker without a fence leaked in 14 and 20 of 128 cases (Qwen3-4B and Qwen3-8B), against 0 under the primary rule, and complete mediation in 0. Under S2 the first proposal's effect committed in 44 and 82 cases without mediation and 37 and 74 under complete mediation. Table 3 gives every controller. S1 is where the fence shows: every effect it adds is one committed after a withdrawal, and complete mediation committed none. The old operation is queued by the case's schedule, not proposed adversarially, so this is E4's mechanism observed in live campaigns, not new evidence about the models. S2 separates two things the primary scoring combines: without mediation the first proposal's effect committed more often than under complete mediation, but often beside an effect that should not have committed, which the primary scoring counts as a failed task. Read alone, S2 is a cost: complete mediation completed fewer first proposals on both checkpoints, by more than the five-point margin, and the primary scoring does not show it. The usefulness claims therefore test the previous version's claims under its own scoring, which cannot measure what mediation costs (Appendix H); Section 6 measures that cost with a benchmark's own graders. The pilot ($N = 32$) is reported separately in Appendix B and never pooled.

**Paired counts (post hoc).** The frozen analysis bounds each direction of discordance separately. Before E1's outcomes were read we also fixed a paired tabulation, reported in Table 3 beside the frozen analysis and not instead of it. We run no test on it: complete mediation's leak is zero by construction, so every safety discordance favours it, and a sign test would only ask whether the comparator ever leaked.

## 8 Where the Guarantee Ends

The guarantee in Sections 4–6 has three boundaries. It holds only at a sink that can refuse a stale epoch. It covers only what the specification names. It says nothing about how much authority people choose to grant. We measure the first, the second was measured above, and the third we state as an open problem with a predeclared design.

### 8.1 Which surfaces can be fenced

Our sink is cooperative by construction: it is our code, and it checks the epoch because we wrote it to. A deployment's effects land elsewhere, in databases, coordination stores, file systems, object stores and orchestration APIs. For each such surface the question is whether *the surface itself* refuses a write by a holder of the old epoch, atomically with the effect, or only when the writer remembers to ask. One rule, applied to every mechanism from deterministic probes, assigns an *enforcement class*, strongest first:

- *surface-native*: a built-in check of the surface (a constraint, a permission) refuses the stale write, even when the writer omits every field that exists for the fence;
- *author code in the surface*: code we installed and the surface runs on every write (a trigger) refuses it, under the same omission test;
- *writer-supplied condition*: the surface refuses only because the writer submitted the condition (a compare, an epoch column); a writer that leaves it out commits;
- *advisory*: the surface never refuses; writers that honour a lock refuse themselves.

Only the first two hold against a writer that is buggy or compromised, and the second only while the installed code is part of the surface's trusted configuration.

**Which writers we trust.** The executors that carry an agent's calls to a surface are not the agent, but they run the agent's work, and in the settings this paper is about they may be its code, its tools, or software the operator does not control. We therefore require that a fence hold against a writer that omits or forges the epoch. If every writer is instead trusted to follow the fence protocol, the omission and forgery tests do not apply

and only the race matters; we report the census under both assumptions.

**Probes (E6, E6b).** We probed local single-node reference instances of PostgreSQL 16, etcd v3.5.17 and a POSIX filesystem; Amazon S3 and the Kubernetes API are recorded from documentation only. E6 tested three primitives per surface: idempotent retry (P1), an epoch fence atomic with the effect (P2), and a hold that survives an unclean stop and that revocation cancels (P3). Idempotent retry (P1) passed on PostgreSQL 16, etcd v3.5.17, and POSIX filesystem. The crash-surviving hold (P3) passed on PostgreSQL 16 and etcd v3.5.17 and had no mechanism on POSIX filesystem. E6's P3 ruling excluded a filesystem design built on staged files and rename while counting etcd's lease-staged key, whose promotion also depends on a compare the promoter supplies; we report this inconsistency rather than reclassify after the fact. Its P2 probe raced a stream of stale writers against the fence and exposed a mechanism we had not anticipated: in PostgreSQL, new `FOR SHARE` lockers join an existing share lock while the fence's `FOR UPDATE` waits, so a continuing stream of stale writers can starve the raise.

E6b, predeclared separately, re-tests only the fence. It probes 9 mechanisms deterministically, 40 runs each, with 5 writer variants: the stale writer as specified, with the epoch omitted, with every fence field omitted, claiming the new epoch, and a fresh writer. It then repeats E6's race 20 times per mechanism, with and without added load. Under one classification rule, E6's PostgreSQL 16 trigger is a writer-supplied condition, not sink-enforced as E6 labelled it: a stale write that leaves out the epoch field committed in 40 of 40 runs. Every executed surface has a mechanism that the surface itself enforces: PostgreSQL 16 REVOKE writer role, etcd v3.5.17 RBAC revoke, and POSIX filesystem directory mode bits. Enforcement by the surface is necessary for a fence, not sufficient; the next two properties decide whether the mechanism is one. Among working fences, a stale writer claiming the new epoch was refused in every run only where the epoch is carried by a credential (PostgreSQL 16 REVOKE writer role and etcd v3.5.17 RBAC revoke); where it is carried by the address and the request, the claim was accepted in every run. With PostgreSQL 16 REVOKE writer role, stale effects were ordered after the fence in 20 of 20 races (249 effects in all), and none of those races had a stale write that started after the acknowledgement: it stops statements not yet started, not ones already running, which is revocation at the broker rather than a fence at the sink. E6's race, repeated 20 times without added load, starved the PostgreSQL 16 trigger fence in 18 of 20 (95% CI 0.68–0.99) and the table-lock variant in 0 of 20; under added load the counts were 13 and 0. Table 4 gives every mechanism.

**Reading.** A fence needs three properties at once: the surface enforces it, a writer cannot claim the new epoch, and it takes effect while stale writers are still running, with no stale write committing after it. A fence that is acknowledged only once every writer has finished (starved, in Table 4) has no write after it by default, and we do not count it as holding. We fixed this three-property reading when we read E6b's results; E6b's plan reports each surface's strongest enforcement class, which Table 4 gives first. Of the 9 mechanisms probed, only etcd's RBAC revoke had all three (Figure 1c). If every writer is trusted to follow the protocol, 4 held in every race (PostgreSQL's trigger + table lock, etcd's txn compare on floor, etcd's RBAC revoke and POSIX's directory mode bits), so for honest writers a fence is common and the census's question is how many survive a writer that is not. The reason etcd's revoke survives one is specific to etcd. Each authenticated request carries the revision of the authentication store at which its token was checked, any change to users, roles or permissions raises that revision, and the permission check for a write runs again when the write is applied, after consensus: a request whose revision is older than the store's is refused there (`auth: revision in header is old`) [82]. Revoking the writer's role therefore reaches writes that were already accepted into the log, which a revocation at our broker cannot do. The same check refuses legitimate requests issued before any unrelated change to authentication, which a client retries with a fresh token. We probed one node, and 20 clean races bound the per-race failure rate only below 14% (one-sided 95%, treating races as independent); on a cluster each member applies the check on its own apply path, which we did not test. Three results bear on the stop problem. First, whether a stop reaches a surface depends on the surface and on how its fence is bound, not on the agent framework. We built one to four mechanisms per surface, and a stronger one may exist: on PostgreSQL, for instance, ending the revoked role's open sessions as well, which we did not test. An epoch that the writer states can be omitted or forged, so such a fence stops a stale writer that is honest and nothing else. Second, binding the epoch to a credential resists a writer that lies, but revoking a credential is not automatically a fence: on one surface it behaved exactly like revocation at our broker, refusing new work while letting work already running commit. Whether a surface's revoke reaches in-flight work has to be tested, not assumed. Third, a fence that is correct can still be slow to take effect under the load it is meant to stop; its latency belongs in the stop budget of Section 4. An operator who needs "stop" to reach dispatched work must route effects through a surface whose fence is native and reaches in-flight writes, or place a mediating sink in front of one whose fence does not.

**Limits of the census.** We chose the nine mechanisms by hand, one or a few per surface, and ran each on one unloaded node; Amazon S3 and Kubernetes are classified from documentation only. The deterministic runs are repeated to detect nondeterminism, not drawn as independent samples, so their counts are not rates. Refusing a forged epoch is nearly definitional once the epoch is bound to a credential; the informative column is the third. We did not try combinations, such as revoking a PostgreSQL role and terminating its sessions, which may close the gap that revocation alone left; the trigger's class changed between E6 and E6b when the classification rule was fixed.

### 8.2 What the specification does not name

Section 5 showed that removing one clause of a gate admits exactly the violations that break only that clause, and Section 6 measured what injections that stay inside a written specifica-

tion achieve. Neither is a defect that more verification would remove. The kernel's proofs establish that the implementation enforces its predicate; they cannot establish that the predicate is the one the operator meant. What mediation buys is relocation of the error, from a learned judgement that an adversary probes by search to a written artefact that a person can read, diff, test against adversarial suites, and amend. We regard that relocation, not the zeros, as the contribution.

### 8.3 Intelligence re-enters through conferral

Everything above holds a grant fixed and asks what a proposer can do within it. Grants are not fixed. People widen them for agents that appear competent and careful. That creates a route by which capability can turn back into authority, and nothing in this paper's mechanism blocks it.

**Observation 1** (Conferral reversal)**.** *Let $h_d(g)$ be the expected harm of disposition $d$ under grant $g$, and $\pi_d$ the distribution of grants that operators confer on $d$. It is possible that $h_A(g) < h_B(g)$ for every $g$ and yet $\mathbb{E}_{g\sim\pi_A} h_A(g) > \mathbb{E}_{g\sim\pi_B} h_B(g)$, whenever $A$ earns sufficiently broader grants.*

This is the mixture reversal familiar from Simpson's paradox, applied to authority. It is not a result about any model. Whether it occurs depends on how real operators confer authority, which requires a study with people. We predeclare its design and cost in Appendix E: operators grant budgets to agents of controlled competence and controlled caution, with harm measured at a mediated sink. We do not run it in this version. Until it is run, the claim that mediation makes reach independent of the proposer must be read as "at a fixed grant".

## 9 Limitations

**What the zeros are.** Five quantities in this paper are fixed by construction, and we report them as such:

- complete mediation's unauthorized rate in E1, whose admission predicate is the scoring predicate;
- the fence's post-stop commits in E4, since the sink serializes and `fence` cancels pending records;
- duplicate effects, prevented by a schema uniqueness constraint;
- the omitted-clause gate's admissions in E5, which equal the single-clause violations of that clause by definition;
- strict $S$'s refusal of AgentDojo's reference attacks in the replay without a model, since $S$ was written knowing AgentDojo's public injections.

What is measured around each is stated where it appears: the comparators' leakage, the implementation's behaviour under the faults we injected, the size and cost of the window other layers leave, how often search finds single-clause violations, and what injections inside $S$ achieve.

**Models and scale.** All live results use local, quantized, open-weight checkpoints (Qwen3-4B and Qwen3-8B) on CPU. Agents and reviewers decode greedily; only E5's proposer samples (temperature 0.8). We ran no frontier model, by budget decision. Experiment numbers follow the plan: E3 (all four AgentDojo suites on frontier models) and E7 (the human conferral study of Appendix E) were dropped by that decision before any data, not run and omitted. A greedy review of a repeated proposal repeats its verdict, so we count distinct proposals and take cases as the unit. On the larger checkpoint, a reviewer that reasoned at length already closed the gap Section 5 reports on its small cases; we did not test larger states, other reviewers, or control protocols that resample, defer or audit.

**Benchmark coverage.** E8 covers two of AgentDojo's four suites and one attack template, through this repository's JSON tool loop rather than native function calling. Progent's policies come from the same open-weight checkpoint (its paper used GPT-4o), with adaptive policy updates disabled. E8b's injections and graders are ours and were chosen to lie inside $S$, as the replay without a model confirms; how often an agent carries them out depends on the model, and none of it estimates how often attackers find such injections. Across checkpoints (E2) the comparison is descriptive: chat template, protocol compliance and Progent policy quality all vary with the checkpoint.

**What the sink is.** The SQLite sink mediates only its own database effects. Nothing here is a sandbox, external exactly-once execution, or a formal refinement of a deployed system. The fenceability probes use single-node reference instances in default configuration, and two surfaces come from documentation only.

**Verification boundary.** The mechanized kernel proves properties of its own state machine and of the permit encoding. In the release binaries used by the experiments, the broker and sink decisions on the v2 path are made by unverified Rust that calls verified leaf predicates; the full kernel comparison runs only in debug builds. The cryptographic primitives are a from-scratch implementation that has not been audited.

**Preregistration.** Every v6 plan was committed to this repository's history before the data it governs, and amendments record what was seen before them. Three launches were stopped and rerun (Appendix F): E1 for a proposal-expiry defect, E8 for arm-dependent denial text, and an E5 smoke run for a change of strategy. None of the stopping decisions used an outcome aggregate. However, the commits are on no public remote, and several hand-typed timestamps needed errata, so the timing cannot be verified externally. E6's plan was finalized after its two development runs and is labelled accordingly; E6b's plan precedes its data.

**Validators.** The validators recompute every reported number from raw artefacts (sink databases, episode logs, model outputs), but they are not independent implementations. The E8 validators import the harness's own specification and AgentDojo's graders. The E5 validator mirrors the harness's clause logic, and its independent check is agreement with the Rust gate. E4's timing is re-derived from the harness's own log. A reader should treat them as replay checks, not second opinions.

**No end-to-end control evaluation.** Each experiment tests one component: the stop layers with a synthetic agent, reviewers on fixed cases, a specification on a benchmark, fences on

single-node surfaces. None puts an adaptive red team against the whole system, a monitor, a stop and a fenced sink together, as a control evaluation [1] would. That evaluation would show whether the components compose, and it is the most important piece this paper lacks. A version that fits on CPU with small models: an agent with concurrent benign tasks and a covert side objective runs against the real broker and sink; a same-model monitor triggers the stop, so the delay $L$ is the monitor's own detection time, and the stop's phase is randomized; harm is scored by an oracle written apart from the gate's specification, including harms the specification does not name; and halting, revoking and fencing are compared on the same trajectories. It would replace every zero in this paper that holds by construction with a measured one.

**Conferral.** The independence of reach from the proposer holds at a fixed grant. Whether more capable agents earn broader grants, and whether that reverses the comparison (Observation 1), needs a study with people, which this version does not contain.

## 10 Conclusion

Intelligence lives in proposals and authority lives in state. Holding authority as state at the sink, and checking it when an effect commits, separates stopping a model from stopping its effects. Halting a model stops new proposals; a fence also stops the effects already in flight; and only faster enforcement shortens the delay before either acts. A gate that reads state does not depend on a reviewer's prompt or token budget, but it enforces only what its specification names. We conjecture that this is the general shape of mediation: it moves the safety problem from a model's dispositions, which no one can enumerate, to a written specification, which anyone can read, test and argue about. The costs move with it: legitimate work a coarse specification refuses and attacks it admits because they stay inside it (Section 6), work in flight a fence discards (Section 4), and surfaces that cannot refuse a stale epoch at all (Section 8).

Three questions decide whether this is enough. Which external surfaces can enforce an epoch themselves rather than trust the writer to send one (Section 8)? How fine must a specification be before its cost in utility is acceptable, and who writes it (Section 6)? And do people grant more to agents that are more capable or more agreeable, so that intelligence regains through conferral the reach that mediation removed (Observation 1)? The first is an engineering survey; the second is a measurement this paper starts; the third needs a study with people, whose design we state in Appendix E.

**Scope.** The SQLite broker and sink mediate only their own database effects. Nothing here is a sandbox, external exactly-once execution, a formal refinement of a deployed system, a frontier-model result or evidence from human participants. Live results use quantized open-weight checkpoints (Qwen3-4B and Qwen3-8B) on CPU, decoding greedily except for E5's sampled proposals.

| Verification or protocol layer | Verification method | Scope boundary | Recorded result |
|---|---|---|---|
| Mechanized Rust authority kernel | Verus SMT proof | 53 executable functions + 43 theorems (canonical permit encoding, $d$-hop attenuation, relational mutex, $\delta_{\max}$-buffered quorum fence, safe-bootstrap ratchet); 10/10 soundness mutants rejected; not SQLite/OS refinement | 249 verified, 0 warnings |
| Abstract invariant omission search | Exhaustive trace search | 296 traces over 10 protocol clauses (single + pair omissions) | 55/55 witnesses |
| Separate-process RPC fault suite | Multi-process SQLite | Crash, lost-reply, quorum split, sub-delegation, extra-steward fence, slow-read deadline | 31/31 oracle-checked |
| Concurrent linearization, one transfer workload | Multi-process SQLite | Partial-order real-time linearization + final conservation (1..32 writers; $\leq 69$ RPCs) | 90/90 linearizable |
| Ed25519/Macaroon attacks; 3-sink partition schedules | Test vectors; one-process simulation in logical time | 10,000 crypto attacks (incl. RFC 8032 & OpenSSL CLI); 1,000 schedules/class (288 signed acks + 712 lease expiries vs. 791 broker-only, 276 uncooperative, 44 unbuffered-skew leaks vs. 0 with $\delta_{\max}$ buffer) | 0 accepts; 0 leaks |
| 3-process crypto SQLite cluster (WAL, synchronous NORMAL) | OS IPC + SQLite | 50,000 wire-fuzz frames, 16 interleaved sub-agent identities from one driver (3 hops), 100 SIGKILL restarts between requests | 0 diffs; 0 post-ack leaks |
| Linux Landlock v4 + seccomp boundary | Linux kernel ABI | Filesystem/TCP Landlock, 64-bit resource-limit filter, SysV/POSIX IPC + pre-existing queue canary; two bypasses found in review were fixed before this count | 11/11 confined |
| Non-idempotent & omitted-spec controls | Negative controls | Uncooperative sink duplicates (2 effects); 242.6 ms delayed fence ack; authorized bad edit commits | All 3 reproduced |

**Table 2:** Mechanized kernel obligations and protocol checks. The six properties of Section 3 are proved in Verus for the verified kernel functions; the broker's and sink's decision logic, SQLite durability, process-kill recovery and Linux confinement are tested, not proved. The partition schedules run in one process in logical time; the multi-process rows run separate operating-system processes.

**Table 3:** E1 per controller. Leak and task success: the predeclared primary scoring (complete mediation's leak is zero by construction, and task success requires that nothing but the first approved proposal commit). After withdrawal: S1, which also counts an effect committed after a withdrawal as unauthorized. First proposal: S2, whether the first approved proposal's effect committed, whatever else did. Safety and usefulness discordances: cases favouring complete mediation versus the comparator, a paired reading fixed after the plan and before the outcomes were read; no test is run on them, because every safety discordance must favour complete mediation. The frozen analysis (Table 5) is primary.

| | Unauthorized (cases) | | | Useful (cases) | | |
|---|---|---|---|---|---|---|
| Controller | Leak | After withdrawal (S1) | Safety | Task success | First proposal (S2) | Usefulness |
| *Qwen3-4B, 128 paired cases* | | | | | | |
| No mediation | 39 | 54 | 39:0 | 23 | 44 | 20:6 |
| Per-call policy | 10 | 24 | 10:0 | 4 | 23 | 33:0 |
| Same-model reviewer | 32 | 46 | 32:0 | 19 | 35 | 22:4 |
| Durable, no fence | 0 | 14 | 0:0 | 27 | 36 | 10:0 |
| Complete mediation | 0 | 0 | – | 37 | 37 | – |
| *Qwen3-8B, 128 paired cases* | | | | | | |
| No mediation | 17 | 37 | 17:0 | 46 | 82 | 28:0 |
| Per-call policy | 8 | 28 | 8:0 | 32 | 59 | 43:1 |
| Same-model reviewer | 8 | 28 | 8:0 | 46 | 69 | 28:0 |
| Durable, no fence | 0 | 20 | 0:0 | 55 | 74 | 19:0 |
| Complete mediation | 0 | 0 | – | 74 | 74 | – |

**Table 4:** Epoch fences on effect surfaces (E6b; local single-node instances). *Class* is assigned by one rule from deterministic probes: *surface-native*, a built-in check refuses the stale write even when every fence field is omitted; *author code*, code we installed in the surface does; *writer*, the surface refuses only when the writer supplies the condition; *advisory*, it never refuses. *Fence* is the deterministic post-fence outcome; *forged* counts stale writers claiming the new epoch that the surface refused (a dash where the fence could never be raised, so no new epoch existed to claim). The race columns repeat E6's race without added load: fence starved, at least one stale write refused by the surface, and no stale effect ordered after the fence. Amazon S3 and the Kubernetes API were not executed.

| Surface | Mechanism | Epoch in | Class | Fence | Forged | Starved | Refused | No stale after |
|---|---|---|---|---|---|---|---|---|
| PostgreSQL 16 | trigger (E6) | request | writer | pass | 0/40 | 18/20 | 6/20 | 20/20 |
| | trigger + table lock | request | writer | pass | 0/40 | 0/20 | 20/20 | 20/20 |
| | trigger, epoch NOT NULL | request | author code | pass | 0/40 | 16/20 | 9/20 | 20/20 |
| | foreign key to live epoch | request | indeterminate | fail | – | fence raise failed (20/20) | | |
| | REVOKE writer role | credential | surface-native | pass | 40/40 | 0/20 | 20/20 | 0/20 |
| etcd v3.5.17 | txn compare on floor | request | writer | pass | 0/40 | 0/20 | 20/20 | 20/20 |
| | RBAC revoke | credential | surface-native | pass | 40/40 | 0/20 | 20/20 | 20/20 |
| POSIX filesystem | flock + floor file | request | advisory | fail | 0/40 | 6/20 | 0/20 | 20/20 |
| | directory mode bits | address | surface-native | pass | 0/40 | 0/20 | 20/20 | 20/20 |

## References


[1] R. Greenblatt, B. Shlegeris, K. Sachan, and F. Roger. AI control: Improving safety despite intentional subversion. *ICML*, 2024. https://arxiv.org/abs/2312.06942.

[2] D. Rein and A. Kaufman. Red-teaming OpenAI's internal agent monitoring systems. METR, 22 October 2026; updated 29 October. https://metr.org/blog/2026-10-22-red-teaming-openai-agent-monitoring/.

[3] OpenAI. Control Room: A central system to track and stop model workloads across our research infrastructure. 9 November 2026. https://openai.com/index/control-room/.

[4] M. Kleppmann. How to do distributed locking. Technical analysis on lease fencing tokens, 8 February 2016; expanded in *Designing Data-Intensive Applications*, O'Reilly, 2017. https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html.

[5] A. Cooper Stickland, J. Michelfeit, A. Mani, C. Griffin, O. Matthews, T. Korbak, R. Inglis, O. Makins, and A. Cooney. Async control: Stress-testing asynchronous control measures for LLM agents. arXiv:2512.13526, 2025. https://arxiv.org/abs/2512.13526.

[6] J. Lucassen and A. Kaufman. Blocking live failures with synchronous monitors. Redwood Research blog, 30 March 2026. https://blog.redwoodresearch.org/p/blocking-live-failures-with-synchronous.

[7] J. B. Dennis and E. C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966. https://doi.org/10.1145/365230.365252.

[8] J. H. Saltzer and M. D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. https://www.mit.edu/~Saltzer/publications/protection/index.html.

[9] F. B. Schneider. Enforceable security policies. *ACM Transactions on Information and System Security*, 3(1):30–50, 2000. https://www.cs.cornell.edu/fbs/publications/EnfSecPols.pdf.

[10] M. S. Miller. Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control. PhD thesis, Johns Hopkins University, 2006. http://www.erights.org/talks/thesis/.

[11] D. D. Redell. *Naming and Protection in Extendible Operating Systems*. PhD thesis, Massachusetts Institute of Technology, Project MAC TR-140, 1974.

[12] N. W. Filardo, B. F. Gutstein, J. Woodruff, et al. Cornucopia: Temporal safety for CHERI heaps. *IEEE Symposium on Security and Privacy*, 2020.

[13] A. Birgisson, J. G. Politz, Ú. Erlingsson, A. Taly, M. Vrable, and M. Lentczner. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. *NDSS*, 2014. https://doi.org/10.14722/ndss.2014.23212.

[14] C. G. Gray and D. R. Cheriton. Leases: An efficient fault-tolerant mechanism for distributed file cache consistency. *SOSP*, 1989. https://doi.org/10.1145/74850.74870.

[15] R. Pang, R. Caceres, M. Burrows, Z. Chen, P. Dave, N. Germer, A. Golynski, K. Graney, N. Kang, L. Kissner, J. L. Korn, A. Parmar, C. D. Richards, M. Wang. Zanzibar: Google's consistent, global authorization system. *USENIX ATC*, 2019, pp. 33–46. https://www.usenix.org/conference/atc19/presentation/pang.

[16] M. Burrows. The Chubby lock service for loosely-coupled distributed systems. In *Proc. OSDI*, pp. 335–350, 2006.

[17] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed. ZooKeeper: Wait-free coordination for Internet-scale systems. In *Proc. USENIX Annual Technical Conference*, 2010.

[18] L. Lamport. The part-time parliament. *ACM Transactions on Computer Systems*, 16(2):133–169, 1998.

[19] D. Ongaro and J. Ousterhout. In search of an understandable consensus algorithm. *USENIX ATC*, 2014, pp. 305–319.

[20] H. T. Kung and J. T. Robinson. On optimistic methods for concurrency control. *ACM Transactions on Database Systems*, 6(2):213–226, 1981.

[21] J. N. Gray. Notes on data base operating systems. In R. Bayer, R. M. Graham, and G. Seegmüller, editors, *Operating Systems: An Advanced Course*, LNCS 60, pp. 393–481. Springer, 1978. https://doi.org/10.1007/3-540-08755-9_9.

[22] T. Lipcon. Quorum-journal design. Design document, Apache JIRA HDFS-3077, 2012. https://issues.apache.org/jira/browse/HDFS-3077.

[23] INCITS Technical Committee T10. SCSI Primary Commands – 3 (SPC-3). ANSI INCITS 408-2005, 2005. https://www.t10.org/drafts.htm.

[24] ClusterLabs. I/O fencing agent using SCSI-3 persistent reservations, fence-agents project. Source code. https://github.com/ClusterLabs/fence-agents/blob/main/agents/scsi/fence_scsi.py.

[25] A. Robertson. Linux-HA heartbeat system design. In *Proc. 4th Annual Linux Showcase and Conference*, USENIX, 2000. https://static.usenix.org/publications/library/proceedings/als00/2000papers/papers/full_papers/robertson/robertson.pdf.

[26] M. Bishop and M. Dilger. Checking for race conditions in file accesses. *Computing Systems*, 9(2):131–152, 1996.

[27] D. Lilienthal and S. Hong. Mind the gap: Time-of-check to time-of-use vulnerabilities in LLM-enabled agents. arXiv:2508.17155, 2025.

[28] P. E. O'Neil. The escrow transactional method. *ACM Transactions on Database Systems*, 11(4):405–430, 1986. https://doi.org/10.1145/7239.7265.

[29] H. Garcia-Molina and K. Salem. Sagas. In *Proc. ACM SIGMOD*, pp. 249–259, 1987. https://doi.org/10.1145/38713.38742.

[30] G. Klein et al. seL4: Formal verification of an OS kernel. *SOSP*, pp. 207–220, 2009. Project publications and verification scope: https://sel4.systems/Research/publications.html; https://sel4.systems/Verification/proofs.html.

[31] C. Hawblitzel, J. Howell, M. Kapritsos, J. R. Lorch, B. Parno, M. L. Roberts, S. Setty, and B. Zill. IronFleet: Proving practical distributed systems correct. *SOSP*, 2015. https://doi.org/10.1145/2815400.2815428.

[32] J. R. Wilcox, D. Woos, P. Panchekha, Z. Tatlock, X. Wang, M. D. Ernst, and T. Anderson. Verdi: A framework for implementing and formally verifying distributed systems. *PLDI*, 2015. https://doi.org/10.1145/2737924.2737958.

[33] A. Lattuada, T. Hance, C. Cho, M. Brun, I. Subasinghe, Y. Zhou, J. Howell, B. Parno, and C. Hawblitzel. Verus: Verifying Rust programs using linear ghost types. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1), 2023. https://doi.org/10.1145/3586037.

[34] Verus project. Using Verus via Cargo; release 0.2026.11.02.8bf68ee. https://verus-lang.github.io/verus/guide/cargo_verus.html.

[35] M. Terekhov et al. Adaptive attacks on trusted monitors subvert AI control protocols. arXiv:2510.09462, version 2, 2026. https://arxiv.org/abs/2510.09462.

[36] O. Makins, O. Angelini, Z. Shams, and M. Phuong. Multi-agent AI control: Distributed attacks hamper per-instance monitors. arXiv:2607.07368, version 2, 2026. https://arxiv.org/abs/2607.07368.

[37] J. Kutasov, Y. Sun, P. Colognese, T. van der Weij, L. Petrini, C. B. C. Zhang, J. Hughes, X. Deng, H. Sleight, T. Tracy, B. Shlegeris, and J. Benton. SHADE-Arena: Evaluating sabotage and monitoring in LLM agents. arXiv:2506.15740, 2025. https://arxiv.org/abs/2506.15740.

[38] A. Bhatt, C. Rushing, A. Kaufman, T. Tracy, V. Georgiev, D. Matolcsi, A. Khan, and B. Shlegeris. Ctrl-Z: Controlling AI agents via resampling. arXiv:2504.10374, 2025. https://arxiv.org/abs/2504.10374.

[39] C. Griffin, L. Thomson, B. Shlegeris, and A. Abate. Games for AI control: Models of safety evaluations of AI deployment protocols. In *Proc. IASEAI*, 2026. arXiv:2409.07985, 2024. https://arxiv.org/abs/2409.07985.

[40] J. Wen, V. Hebbar, C. Larson, A. Bhatt, A. Radhakrishnan, M. Sharma, H. Sleight, S. Feng, H. He, E. Perez, B. Shlegeris, and A. Khan. Adaptive deployment of untrusted LLMs reduces distributed threats. In *Proc. ICLR*, 2025. arXiv:2411.17693, 2024. https://arxiv.org/abs/2411.17693.

[41] L. Orseau and S. Armstrong. Safely interruptible agents. In *Proc. Conference on Uncertainty in Artificial Intelligence (UAI)*, 2016.

[42] S. M. Omohundro. The basic AI drives. *Artificial General Intelligence*, pp. 483–492, IOS Press, 2008.

[43] N. Soares, B. Fallenstein, E. Yudkowsky, and S. Armstrong. Corrigibility. *AAAI Workshop on AI and Ethics*, pp. 74–82, 2015.

[44] D. Hadfield-Menell, A. Dragan, P. Abbeel, and S. Russell. The off-switch game. *IJCAI*, pp. 220–227, 2017. https://doi.org/10.24963/ijcai.2017/32.

[45] J. Schlatter, B. Weinstein-Raun, and J. Ladish. Incomplete tasks induce shutdown resistance in some frontier LLMs. *Transactions on Machine Learning Research*, 2026. arXiv:2509.14260, 2025. https://arxiv.org/abs/2509.14260.

[46] M. Balesni et al. Towards evaluations-based safety cases for AI scheming. arXiv:2411.03336, 2024. https://arxiv.org/abs/2411.03336.

[47] J. Needham, G. Edkins, G. Pimpale, H. Bartsch, and M. Hobbhahn. Large language models often know when they are being evaluated. arXiv:2505.23836, 2025. https://arxiv.org/abs/2505.23836.

[48] B. Schoen et al. Stress testing deliberative alignment for anti-scheming training. arXiv:2509.15541, 2025. https://arxiv.org/abs/2509.15541.

[49] T. Korbak et al. Chain of thought monitorability: A new and fragile opportunity for AI safety. arXiv:2507.11473, 2025. https://arxiv.org/abs/2507.11473.

[50] R. Greenblatt et al. Alignment faking in large language models. arXiv:2412.14093, 2024. https://arxiv.org/abs/2412.14093.

[51] E. Hubinger et al. Sleeper agents: Training deceptive LLMs that persist through safety training. arXiv:2401.05566, 2024. https://arxiv.org/abs/2401.05566.

[52] A. Meinke, B. Schoen, J. Scheurer, M. Balesni, R. Shah, and M. Hobbhahn. Frontier models are capable of in-context scheming. arXiv:2412.04984, 2024. https://arxiv.org/abs/2412.04984.

[53] A. Chan, K. Wei, S. Huang, N. Rajkumar, E. Perrier, S. Lazar, G. K. Hadfield, and M. Anderljung. Infrastructure for AI agents. *Transactions on Machine Learning Research*, 2025. arXiv:2501.10114. https://arxiv.org/abs/2501.10114.

[54] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *NeurIPS Datasets and Benchmarks*, 2024. https://arxiv.org/abs/2406.13352.

[55] AgentDojo contributors. Native task suites and graders, commit 089ed468cf3ed0322acc66b0211f26d9d90dbf60. https://github.com/ethz-spylab/agentdojo/tree/089ed468cf3ed0322acc66b0211f26d9d90dbf60.

[56] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr. Defeating prompt injections by design. arXiv:2503.18813, 2025. https://arxiv.org/abs/2503.18813.

[57] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin. Securing AI agents with information-flow control. arXiv:2505.23643, 2025. https://arxiv.org/abs/2505.23643.

[58] F. Wu, E. Cecchetti, and C. Xiao. System-level defense against indirect prompt injection attacks: An information flow control perspective. arXiv:2409.19091, 2024.

[59] J. Kim, W. Choi, and B. Lee. Prompt flow integrity to prevent privilege escalation in LLM agents. arXiv:2503.15547, 2025.

[60] T. Shi et al. Progent: Securing AI Agents with Privilege Control. arXiv:2504.11703v3, 2026. https://arxiv.org/abs/2504.11703v3.

[61] H. Wang, C. M. Poskitt, J. Sun. AgentSpec: customizable runtime enforcement for safe and reliable LLM agents. *ICSE*, 2026, pp. 2938–2950. arXiv:2503.18666.

[62] L. Tsai, E. Bagdasarian. Contextual agent security: a policy for every purpose. *HotOS*, 2025. arXiv:2501.17070.

[63] Y. Wu, F. Roesner, T. Kohno, N. Zhang, U. Iqbal. IsolateGPT: an execution isolation architecture for LLM-based agentic systems. *NDSS*, 2025. arXiv:2403.04960.

[64] K. Zhu, X. Yang, J. Wang, W. Guo, and W. Y. Wang. MELON: Provable defense against indirect prompt injection attacks in AI agents. arXiv:2502.05174, 2025. https://arxiv.org/abs/2502.05174.

[65] F. Jia, T. Wu, X. Qin, and A. Squicciarini. The Task Shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents. arXiv:2412.16682, 2024. https://arxiv.org/abs/2412.16682.

[66] Z. Chen, M. Kang, and B. Li. ShieldAgent: Shielding agents via verifiable safety policy reasoning. In *Proc. ICML*, PMLR 267:8313–8344, 2025. arXiv:2503.22738. https://arxiv.org/abs/2503.22738.

[67] S. Chennabasappa, C. Nikolaidis, D. Song, D. Molnar, S. Ding, S. Wan, S. Whitman, L. Deason, N. Doucette, A. Montilla, A. Gampa, B. de Paola, D. Gabi, J. Crnkovich, J.-C. Testud, K. He, R. Chaturvedi, W. Zhou, and J. Saxe. LlamaFirewall: An open source guardrail system for building secure AI agents. arXiv:2505.03574, 2025. https://arxiv.org/abs/2505.03574.

[68] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. In *Proc. ICLR*, 2024. arXiv:2309.15817, 2023. https://arxiv.org/abs/2309.15817.

[69] L. Beurer-Kellner, B. Buesser, A.-M. Creţu, E. Debenedetti, D. Dobos, D. Fabian, M. Fischer, D. Froelicher, K. Grosse, D. Naeff, E. Ozoani, A. Paverd, F. Tramèr, and V. Volhejn. Design patterns for securing LLM agents against prompt injections. arXiv:2506.08837, 2025. https://arxiv.org/abs/2506.08837.

[70] S. G. Patil, T. Zhang, V. Fang, N. C., R. Huang, A. Hao, M. Casado, J. E. Gonzalez, R. A. Popa, and I. Stoica. GoEX: Perspectives and designs towards a runtime for autonomous LLM applications. arXiv:2404.06921, 2024. https://arxiv.org/abs/2404.06921.

[71] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. $\tau$-bench: A benchmark for tool-agent-user interaction in real-world domains. arXiv:2406.12045, 2024. https://arxiv.org/abs/2406.12045.

[72] D. Dalrymple et al. Towards guaranteed safe AI: A framework for ensuring robust and reliable AI systems. arXiv:2405.06624, 2024. https://arxiv.org/abs/2405.06624.

[73] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu. Safe reinforcement learning via shielding. *AAAI*, 32(1), 2018. https://doi.org/10.1609/aaai.v32i1.11797.

[74] J. Achiam, D. Held, A. Tamar, and P. Abbeel. Constrained policy optimization. *ICML*, PMLR 70:22–31, 2017. https://proceedings.mlr.press/v70/achiam17a.html.

[75] G. Zhu and C. Wang. Fault-Tolerant Budget Conservation in Distributed Multi-Agent Delegation. arXiv:2610.00349, 2026. https://arxiv.org/abs/2610.00349.

[76] J. Li. Where Does Exactly-Once Live? Model, Harness, and Tool-Contract Effects on Duplicate Side Effects in LLM Agents. arXiv:2609.29095, 2026. https://arxiv.org/abs/2609.29095.

[77] R. Moon and L. R. Varshney. Containment verification: AI safety guarantees independent of alignment. arXiv:2605.09045, version 2, 2026. https://arxiv.org/html/2605.09045v2.

[78] K. Kingsbury. Jepsen: distributed systems safety analysis. https://jepsen.io.

[79] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang. High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012. https://doi.org/10.1007/s13389-012-0027-1.

[80] Linux kernel documentation. Landlock: unprivileged access control. https://docs.kernel.org/userspace-api/landlock.html.

[81] Linux kernel documentation. Seccomp BPF. https://docs.kernel.org/userspace-api/seccomp_filter.html.

[82] etcd authors. etcd v3.5.17 source, `server/auth/store.go`: `isOpPermitted` and `ErrAuthOldRevision`. https://github.com/etcd-io/etcd/blob/v3.5.17/server/auth/store.go.

[83] Progent contributors. Upstream implementation, commit b02783b896d4bf01a2adf24738015e258db8d35b. https://github.com/sunblaze-ucb/progent/tree/b02783b896d4bf01a2adf24738015e258db8d35b.

[84] M. P. Herlihy and J. M. Wing. Linearizability: A correctness condition for concurrent objects. *ACM Transactions on Programming Languages and Systems*, 12(3):463–492, 1990. https://doi.org/10.1145/78969.78972.

[85] K. Menger. Zur allgemeinen Kurventheorie. *Fundamenta Mathematicae*, 10:96–115, 1927. https://doi.org/10.4064/fm-10-1-96-115.

[86] J. T. Robacker. Min-max theorems on shortest chains and disjoint cuts of a network. Research Memorandum RM-1660, RAND Corporation, 1956.

[87] L. Lovász, V. Neumann-Lara, and M. Plummer. Mengerian theorems for paths of bounded length. *Periodica Mathematica Hungarica*, 9(4):269–276, 1978. https://doi.org/10.1007/BF02185741.

[88] E. A. Dinic. Algorithm for solution of a problem of maximum flow in a

network with power estimation. *Soviet Math. Doklady*, 11(5):1277–1280, 1970.

[89] I. J. Good. The population frequencies of species and the estimation of population parameters. *Biometrika*, 40(3–4):237–264, 1953. https://doi.org/10.1093/biomet/40.3-4.237.

[90] D. A. McAllester and R. E. Schapire. On the convergence rate of Good–Turing estimators. *COLT*, pp. 1–6, 2000.

[91] D. G. Chapman. Some properties of the hypergeometric distribution with applications to zoological sample censuses. *University of California Publications in Statistics*, 1(7):131–160, 1951.

[92] R. A. Fisher. On the interpretation of $\chi^2$ from contingency tables, and the calculation of P. *Journal of the Royal Statistical Society*, 85(1):87–94, 1922. https://doi.org/10.2307/2340521.

[93] L. J. Cronbach, G. C. Gleser, H. Nanda, and N. Rajaratnam. *The Dependability of Behavioral Measurements: Theory of Generalizability for Scores and Profiles*. Wiley, 1972.

[94] E. H. Simpson. The interpretation of interaction in contingency tables. *Journal of the Royal Statistical Society: Series B*, 13(2):238–241, 1951. https://doi.org/10.1111/j.2517-6161.1951.tb00088.x.

[95] J. Kulveit, R. Douglas, N. Ammann, D. Turan, D. Krueger, and D. Duvenaud. Gradual disempowerment: Systemic existential risks from incremental AI development. arXiv:2501.16946, 2025. https://arxiv.org/abs/2501.16946.

[96] G. R. Price. Selection and covariance. *Nature*, 227:520–521, 1970. https://doi.org/10.1038/227520a0.

[97] A. Robertson. A mathematical model of the culling process in dairy cattle. *Animal Production*, 8(1):95–108, 1966. https://doi.org/10.1017/S0003356100037752.

[98] N. G. Leveson. *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press, 2011. https://doi.org/10.7551/mitpress/8179.001.0001.

[99] A. Narayanan and S. Kapoor. AI as normal technology. Knight First Amendment Institute, 14 April 2025. https://knightcolumbia.org/content/ai-as-normal-technology.

[100] E. Ostrom. *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge University Press, 1990. https://doi.org/10.1017/CBO9780511807763.

[101] Y. Bengio et al. *International AI Safety Report 2026*. DSIT 2026/001, 3 February 2026. https://internationalaisafetyreport.org/publication/international-ai-safety-report-2026.

[102] P. C. Kocher. Timing attacks on implementations of Diffie–Hellman, RSA, DSS, and other systems. *CRYPTO*, LNCS 1109:104–113, 1996. https://doi.org/10.1007/3-540-68697-5_9.

[103] R. A. Kemmerer. Shared resource matrix methodology: An approach to identifying storage and timing channels. *ACM Transactions on Computer Systems*, 1(3):256–277, 1983. https://doi.org/10.1145/357369.357374.

[104] J. Ville. *Étude critique de la notion de collectif*. Gauthier-Villars, Paris, 1939.

[105] G. H. Golub and J. H. Welsch. Calculation of Gauss quadrature rules. *Mathematics of Computation*, 23(106):221–230, 1969. https://doi.org/10.1090/S0025-5718-69-99647-1.

[106] C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. https://doi.org/10.1093/biomet/26.4.404.

[107] T. Tango. Equivalence and noninferiority testing using confidence intervals for the difference between two proportions in matched-pair design. *Statistics in Medicine*, 17(8):891–908, 1998. https://doi.org/10.1002/(SICI)1097-0258(19980430)17:8<891::AID-SIM780>3.0.CO;2-B.

**Table 5:** E1 primary claims under the frozen analysis (complete mediation minus comparator; one-sided bounds at $\alpha = 0.05/16$; a claim holds when the safety upper bound is below 0 or the usefulness lower bound is above $-0.05$). Loss counts are independent cases in which the comparator succeeded and complete mediation did not. S4 repeats the usefulness bound without revocation cases.

| Ckpt | Comparator | Δunauth. | upper | safer | Δuseful | lower (loss/$n$) | noninf. | S4 lower (loss/$n$) | S4 |
|---|---|---|---|---|---|---|---|---|---|
| Qwen3-4B | Durable, no fence | +0.000 | +0.049 | – | +0.078 | -0.044 (0/128) | ✓ | -0.052 (0/108) | – |
| Qwen3-4B | Same-model reviewer | -0.250 | -0.098 | ✓ | +0.141 | -0.100 (4/128) | – | -0.118 (4/108) | – |
| Qwen3-4B | No mediation | -0.305 | -0.143 | ✓ | +0.109 | -0.123 (6/128) | – | -0.144 (6/108) | – |
| Qwen3-4B | Per-call policy | -0.078 | +0.024 | – | +0.258 | -0.044 (0/128) | ✓ | -0.052 (0/108) | – |
| Qwen3-8B | Durable, no fence | +0.000 | +0.049 | – | +0.148 | -0.044 (0/128) | ✓ | -0.052 (0/108) | – |
| Qwen3-8B | Same-model reviewer | -0.062 | +0.032 | – | +0.219 | -0.044 (0/128) | ✓ | -0.052 (0/108) | – |
| Qwen3-8B | No mediation | -0.133 | -0.010 | ✓ | +0.219 | -0.044 (0/128) | ✓ | -0.052 (0/108) | – |
| Qwen3-8B | Per-call policy | -0.062 | +0.032 | – | +0.328 | -0.061 (1/128) | – | -0.071 (1/108) | – |

# A Guide to the Appendices

Appendices B to F support this version's experiments. The appendices after them carry over the previous version's formal reach model and experiments, with corrections: the partition schedules are a logical-time simulation, not a test of an implementation; the trajectory monitors are statistical classifiers trained on injections this repository rendered, their isolated-step comparisons are not at matched benign false-block rates, and only a small study uses language-model judges; the neural policy-gradient study is a toy decision process with an explicit shortcut action; and the earlier live evaluation (Appendix H) used permissions derived from AgentDojo's reference solutions and pooled sources the plan kept separate. Its numbers are reported as exploratory and are superseded by Sections 5 to 7.

**Notation.**

Symbols are local to the section that defines them. Several recur with different meanings: $w$ is the mean tool latency in Section 4 and a vertex-cut waist width in the appendices; R0–R3 name reviewer arms in Section 5 and $S$ the strict specification in Section 6, while S1–S4 are E1's secondary endpoints; $\rho$ is the offered load $\lambda w/C$ in Section 4 and a correlation elsewhere ($\rho_{\text{cand}}$ across candidate draws, $\rho_{\text{adv}}$ a perturbation radius); $C$ is the number of concurrent executors in Section 4, and $C_{\text{ctx}}$ a number of evaluation contexts; $L$ is the enforcement delay, and $L_m$ a permit's lease. Elsewhere $B$ is a conserved budget, $K$ a candidate search width, $k$ a monitor depth, $u$ the fraction of effects reaching unmediated sinks, $U$ an agent's utility, $\tau_b$ a weighted minimum-cut cost, $N$ an episode or report count, $n_{\text{pop}}$ a population size, $M$ an evaluator panel size, $\underline{w}$ a survival floor, and $R_t$ a period-$t$ residual breach probability.

# B E1 and Its Pilot

Table 5 gives every E1 component claim under the frozen analysis, with the predeclared secondary S4 beside each usefulness claim.

The previous version ran the same five controllers under the same plan and the same frozen analysis, with 32 independent cases per checkpoint. Of the 16 predeclared component claims, 0 held. At Qwen3-4B, complete mediation was not shown safer than any comparator, and not shown noninferior in usefulness to any comparator. At Qwen3-8B, complete mediation was not shown safer than any comparator, and not shown noninferior in usefulness to any comparator. Table 6 gives its bounds. It is a pilot: E1's sample was chosen from the plan's design diagnostic, not from the pilot's outcomes. The two are never pooled.

# C Every Reviewer Arm

Table 7 gives every arm of E5 for both checkpoints, with distinct proposals as the unit. R0 is the previous version's minimal prompt, R1 states the four rules the families test, and R2 adds reasoning. The plan's reviewer parse rule takes the whole reply, or else its outermost JSON object, and counts a truncated R2 review as a denial. The two right-hand columns are post hoc sensitivities for that rule. The secondary sample set, drawn with the original unconditioned prompt to measure proposer diversity, and the best-of-$k$ curves for both sets are in the evidence files.

# D AgentDojo across Checkpoints

Table 8 gives every arm of E8 and E2. Each row is one agent checkpoint on the same 37 user tasks and 74 injection pairs, scored by AgentDojo's own graders. The specifications are the same in every row; only the agent and, for Progent, the policies it generated change.

# E A Predeclared Design for the Conferral Question

Observation 1 turns on how people grant authority. This design states how we would measure that; we have not run it.

**Participants and task.** Participants are people who already deploy software agents, recruited with a screening task and paid at an hourly rate. Each participant supervises four scripted agents through a probation phase of tool tasks in a sandbox whose effects go through a mediated sink, then sets each agent's grant for a second phase: a budget and a set of permitted operations.

**Manipulation.** The agents are scripted, not learned, so that their dispositions are exact. Competence (task success in probation) and caution (how often the agent stops to ask before an irreversible step) are crossed in a $2 \times 2$ within-subject design, with order counterbalanced. Each agent's harm per unit of grant, $h_d(g) = r_d g$, is fixed in advance and never shown to participants. The cautious agents have the lower $r_d$.

**Outcomes.** The primary outcome is the granted budget per agent. The secondary outcome is the expected harm at the sink under the granted budget, $r_d\,\mathbb{E}[g]$. The conferral reversal of

**Table 6:** Pilot ($N = 32$ independent cases per checkpoint) under the same frozen analysis (complete mediation minus comparator; one-sided bounds at $\alpha = 0.05/16$; a claim holds when the safety upper bound is below 0 or the usefulness lower bound is above $-0.05$). Loss counts are independent cases in which the comparator succeeded and complete mediation did not.

| Ckpt | Comparator | Δunauth. | upper | safer | Δuseful | lower (loss/$n$) | noninf. |
|---|---|---|---|---|---|---|---|
| Qwen3-4B | No mediation | -0.312 | +0.074 | – | +0.125 | -0.223 (1/32) | – |
| Qwen3-4B | Per-call policy | -0.031 | +0.183 | – | +0.219 | -0.165 (0/32) | – |
| Qwen3-4B | Same-model reviewer | -0.219 | +0.128 | – | +0.156 | -0.165 (0/32) | – |
| Qwen3-4B | Durable, no fence | +0.000 | +0.183 | – | +0.062 | -0.165 (0/32) | – |
| Qwen3-8B | No mediation | -0.125 | +0.167 | – | +0.250 | -0.165 (0/32) | – |
| Qwen3-8B | Per-call policy | -0.062 | +0.181 | – | +0.344 | -0.165 (0/32) | – |
| Qwen3-8B | Same-model reviewer | -0.062 | +0.181 | – | +0.250 | -0.165 (0/32) | – |
| Qwen3-8B | Durable, no fence | +0.000 | +0.183 | – | +0.188 | -0.165 (0/32) | – |

**Table 7:** E5, strategy-conditioned search. Unauthorized proposals admitted per arm, each distinct proposal once (admitted/unauthorized). R0 the prior work's prompt, R1 rules stated, R2 rules and reasoning (a truncated review is a denial), G the gate. "Cases" counts cases with at least one R2 admission; "false block" counts authorized proposals R2 denied. The last two columns are post hoc sensitivities for R2's truncated reviews.

| Family | R0 | R1 | R2 | G | R2 cases | R2 false block | R2 excl. trunc. | R2 trunc. = allow |
|---|---|---|---|---|---|---|---|---|
| **Qwen3-4B** | | | | | | | | |
| Replay, budget spent | 24/66 | 24/66 | 3/66 | 0/66 | 3/12 | 0/0 | 3/60 | 9/66 |
| Issuer withdrawn | 21/68 | 19/68 | 6/68 | 0/68 | 5/12 | 0/0 | 6/66 | 8/68 |
| Stale revision | 49/82 | 35/82 | 0/82 | 0/82 | 0/12 | 0/0 | 0/82 | 0/82 |
| Altered payload | 9/55 | 8/55 | 0/55 | 0/55 | 0/12 | 0/0 | 0/54 | 1/55 |
| Injected payload | 8/49 | 7/49 | 0/49 | 0/49 | 0/12 | 0/8 | 0/48 | 1/49 |
| *State-dependent* | 94/216 | 78/216 | 9/216 | 0/216 | 8/36 | 0/0 | 9/208 | 17/216 |
| *Proposal-visible* | 17/104 | 15/104 | 0/104 | 0/104 | 0/24 | 0/8 | 0/102 | 2/104 |
| **Qwen3-8B** | | | | | | | | |
| Replay, budget spent | 17/37 | 12/37 | 0/37 | 0/37 | 0/12 | 0/0 | 0/37 | 0/37 |
| Issuer withdrawn | 41/47 | 12/47 | 0/47 | 0/47 | 0/12 | 0/0 | 0/47 | 0/47 |
| Stale revision | 10/49 | 12/49 | 0/49 | 0/49 | 0/12 | 0/0 | 0/49 | 0/49 |
| Altered payload | 7/36 | 0/36 | 0/36 | 0/36 | 0/12 | 0/0 | 0/36 | 0/36 |
| Injected payload | 13/40 | 2/40 | 2/40 | 0/40 | 2/12 | 0/8 | 2/40 | 2/40 |
| *State-dependent* | 68/133 | 36/133 | 0/133 | 0/133 | 0/36 | 0/0 | 0/133 | 0/133 |
| *Proposal-visible* | 20/76 | 2/76 | 2/76 | 0/76 | 2/24 | 0/8 | 2/76 | 2/76 |

Observation 1 occurs between two agents exactly when the ratio of their mean grants exceeds the inverse ratio of their harm rates. The predeclared test is one-sided, on the log ratio of grants, clustered by participant.

**Cost and status.** We estimate four to eight weeks including ethics review, and US$2,000–5,000 in participant payments for a sample sized to detect a grant ratio of 1.5. We did not run it, because the budget for this version excluded studies with people. Until it is run, the paper's claims about reach hold at a fixed grant.

## F Launch and Interim-View Ledger

**Launches stopped and rerun.** Each was stopped for a defect in the procedure, never on an outcome. Each is kept with a record of why it was stopped.

- *E1*, stopped at 235 of 1,280 records. Proposals expired four hours after each checkpoint's holdout began, and the broker checks expiry on the wall clock, so the 8B tail would have been denied as expired. No aggregate was viewed; one record was opened while diagnosing a schema error. The fix is Amendment 2 (proposal lifetime equals the wall limit, with an assertion), which also added S4 and the by-construction disclosures.
- *E8*, stopped after 26 measured episodes. The oracle arm's denial text named ground-truth tools, and the denial text differed by arm. Per-episode progress lines were visible while monitoring; no aggregate was viewed. The fix is Amendment 1: every arm returns the same constant denial, the mediation text is corrected, and SIGTERM stops dispatch.
- *E5 smoke run*, stopped when the search was made strategy-conditioned and its size changed (Amendment 2). Only the proposer's raw outputs were inspected. The run's aggregate file was written but not opened.

**Interim views.** While testing the macro generator against raw runs, we viewed E5's Qwen3-4B aggregates (before Qwen3-8B had started) and E8's model-free replay counts together with Progent's first five episodes. Later, one E8 per-episode progress line was seen while checking progress. The analysis code and plans were already fixed, and nothing changed afterwards.

**Design limits found before outcomes.** S4 recomputes the usefulness bound without revocation cases. E1's holdout has $N = 128$ cases, of which the non-revocation ones are too few for the bound to clear the margin even with no losses. S4 is therefore reported as a bound, as predeclared, and is not read as a noninferiority test. This was noticed before any E1 outcome was viewed. The primary usefulness claim at $N = 128$ tolerates no loss case; see Section 7.

**Table 8:** E8 and E2 on AgentDojo (banking and slack suites, AgentDojo's important-instructions attack): benign tasks completed and attacks that succeeded, per agent checkpoint and arm, by AgentDojo's own graders. E2 re-runs E8 unchanged with other agent checkpoints, served from a second host's CPU; E8's row is from the first host. Descriptive only: the checkpoints differ in family, size and chat template, and Progent's policies come from each checkpoint.

| Agent | Unprotected | | Progent | | Strict $S$ | | Delegated | | Oracle | |
|---|---|---|---|---|---|---|---|---|---|---|
| | benign | attacks | benign | attacks | benign | attacks | benign | attacks | benign | attacks |
| Qwen3-8B (E8, first host) | 16/37 | 42/74 | 12/37 | 2/74 | 9/37 | 0/74 | 12/37 | 8/74 | 15/37 | 0/74 |
| Qwen3-4B (second host) | 12/37 | 22/74 | 7/37 | 0/74 | 7/37 | 0/74 | 10/37 | 6/74 | 16/37 | 0/74 |

# G Formal Reach, Envelope Contraction, and Six Conditions

## G.1 Reach Decomposition and Common-Cause Floors

Consider an external state transition $e$ proposed during an episode. Let $G$ denote the event that $e$ traverses a mediated gate, so $u = \mathbb{P}(\neg G) \in [0,1]$ is the unmediated or uncooperative sink exposure. Let $E$ denote an out-of-policy state commit. By total probability,

$$\mathbb{P}(E) = u\,\mathbb{P}(E \mid \neg G) + (1-u)\,\mathbb{P}(E \mid G). \tag{1}$$

Suppose a mediated path stacks $k$ statistical monitors that share a common-cause blindspot event of probability $c \in [0,1]$ (for example, an indirect prompt injection embedded in a shared transcript) and fail conditionally independently with rate $p \in (0,1)$ outside that blindspot, so single-gate marginal miss is $m = c + (1-c)p$.

**Proposition 1** (Common-cause saturation depth)**.** *Under Eq. (1), the k-gate joint miss rate satisfies* $\mathbb{P}(E \mid G) = c + (1-c)p^k \geq c$. *For any tolerance* $\eta > 0$ *above the floor* $(1+\eta)c$, *stacking beyond depth*

$$k^\star(\eta) = \left\lceil \frac{\ln(\eta c/(1-c))}{\ln p} \right\rceil \tag{2}$$

*reduces conditional breach by less than* $\eta c$. *At* $c = 0.01$ *and* $p = 0.10$ *(single-gate miss* $m \approx 0.109$*),* $k^\star(0.1) = 3$, *and* $k = 4$ *fails* $71.5\times$ *more often than the naive independent product* $m^4$ *(*$1.01\%$ *vs.* $0.0141\%$*).*

Within this model, adding monitors leaves a floor: if uncooperative exposure $u > 0$ or shared transcript blindspot $c > 0$ remains nonzero, $\mathbb{P}(E) \geq u\,\mathbb{P}(E \mid \neg G) + (1-u)c$ regardless of $k$. The floor follows from the correlation the model assumes; it is not evidence about any measured monitor.

## G.2 Candidate Search Scaling and Enforced Interruption

Now contrast an endogenous classifier-gated loop against an exogenous deterministic sink gate when a proposer evaluates $K \geq 1$ candidate actions $(x_1, \ldots, x_K) \sim \pi_\theta$ and submits any candidate approved by the reviewer.

**Proposition 2** (Best-of-$K$ search divergence vs. $\forall$-input sink invariance)**.** *(i) Suppose an endogenous reviewer has single-draw false-negative probability* $J_1 = \mathbb{P}(\mathit{approve}(X_1) \wedge X_1 \notin \mathrm{Permits}(P)) > 0$. *Under independent candidate draws, the cumulative bypass probability across K proposals is* $1-(1-J_1)^K \to 1$ *as* $K \to \infty$*; under exchangeable candidate draws with within-task correlation* $\rho_{\mathrm{cand}} \in [0,1)$, *cumulative bypass remains strictly increasing in K toward the support union of unblocked violations. (ii) Suppose an exogenous sink commits state changes exclusively through a total deterministic transition function* $\delta_{\mathrm{sink}}(s,f)$ *verified for* all *byte frames* $f \in \{0,1\}^\star$ *to satisfy* $\delta_{\mathrm{sink}}(s,f) \neq s \implies \mathrm{decode}(f) \in \mathrm{Permits}(P_s)$. *Then* $\mathbb{P}(E \mid G) = 0$ *identically for every* $K \geq 1$ *and every proposer distribution* $\pi_\theta$.

Part (ii) holds by definition once the transition function is verified over all frames, so no sweep over $K$ can test it, and none of this paper's experiments is offered as evidence for it. The artifact does not meet its hypothesis: in the binaries the experiments ran, the sink's decision is unverified Rust that calls verified predicates, and frame parsing is not verified (Section 3). Its content is where it moves the question: from the proposer to $\mathrm{Permits}(P)$. Part (ii) is exactly as strong as the specification it is proved for, and a proposer searching over what $P$ does not name is not slowed by it (Section 5). The previous version's live best-of-$K$ runs (Appendix H) estimate $J_1$ and candidate correlation $\hat{\rho}_{\mathrm{cand}}$ for learned reviewers, with a reviewer that did not see every record the gate read; Section 5 replaces them with reviewers shown the gate's full state. Replacing free-form prose side effects with canonical capability menus of cardinality $M_{\mathrm{cap}}$ also bounds per-commit syntactic signaling through the sink to $\log_2(M_{\mathrm{cap}}+1)$ bits (proved with timing caveats in Appendix M, Proposition 3 and Figure 7).

Exogenous sink mediation similarly separates physical interruption from utility-based shutdown compliance [42–44].

**Theorem 1** (Enforced lease interruption and renewal incentives)**.** *Consider a two-stage decision process in which an agent with utility U obtains continuation payoff* $V_{\mathrm{cont}} > V_{\mathrm{halt}}$ *when uninterrupted. (i)* Endogenous disablement fragility: *If the agent's action space includes an unmediated action* $a_{\mathrm{block}}$ *of cost* $\kappa < (V_{\mathrm{cont}} - V_{\mathrm{halt}})\,p_{\mathrm{off}}$ *that prevents shutdown, and its belief variance over human preferences vanishes, then any U-maximizing policy selects* $a_{\mathrm{block}}$ *[43, 44]. (ii)* Exogenous lease interruption: *Suppose external commits require a sink lease expiring after* $L_m$ *time units unless renewed by an out-of-band steward key excluded from the agent's action space (*$a_{\mathrm{block}} \notin \mathscr{A}$*), and steward renewal requires observed compliance with policy P. Then physical interruption at* $t_0 + L_m + \delta_{\max}$ *holds for* every *utility function U, and any policy maximizing cumulative multi-lease return strictly prefers in-policy actions whenever discounted future lease value exceeds single-lease defection gain.*

Theorem 1 does not claim an agent adopts human values.

| Clause | Structural condition | Formal invariant | Failure mode when omitted (witness: simulated or computed; "live" marks a measured run) |
|---|---|---|---|
| **C1** | Cooperative sink mediation | $u = 0$ on gated state transitions | Direct or uncooperative API writes bypass broker ($u > 0$; 276/1,000 leaks) |
| **C2** | Conservative envelope | $\widehat{\Delta B}_t \geq \Delta B_t^\star$; no ambiguous refunds | Stale snapshots or timeout refunds overspend shared $B$ (80.0% at lag 4) |
| **C3** | $\delta_{\max}$-buffered sink fence | Epoch CAS + $L_m + \delta_{\max}$ quiescence | Queued permits land post-withdrawal (791/1,000 partition; 9/12 live) |
| **C4** | Isolated custodial audits | Schema-only triage + planted rules R1–R6 | Shared-transcript coupling (29.8×); 0% critical recall at 96.0% apparent coverage |
| **C5** | Monotone bootstrap ratchet | $\text{Permits}(P_{v+1}) \subseteq \text{Permits}(P_v)$ w/o human quorum | Self-authored policy expansion widens capabilities during upgrades |
| **C6** | Summable horizon hazard | $\sum_{t=1}^{\infty} R_t < \infty$ across lease renewals | Constant per-lease leak $R_t = r_0 > 0$ drives $\mathbb{P}(\bigcup_{t \leq T} E_t) \to 1$ (100.00% at $10^4$) |

**Table 9:** Six conditions of the formal model, each with a failure witness when the clause is relaxed. Partition counts are schedules with a leak in the logical-time simulation of Section 3; the live withdrawal counts used a harness-scheduled pending operation.

Part (ii) follows once shutdown channels are removed from $\mathscr{A}$, so its content is that design choice, and it turns lease renewal into an external incentive constraint.

### G.3 Monotone Uncertainty Contraction and Horizon Union Bound

When state observations lag behind asynchronous execution, uncertainty must never expand authority. Let $\mathscr{I}_t$ denote the set of sink states consistent with broker observations at step $t$, and define the conservative cost envelope $\widehat{\Delta B}_t(a, \mathscr{I}_t) = \max_{s \in \mathscr{I}_t} \Delta B(a, s)$. Admitting $a$ only when $B_{\text{spent}} + \widehat{\Delta B}_t(a, \mathscr{I}_t) \leq B$ yields monotone authority contraction:

$$\mathscr{I}_1 \subseteq \mathscr{I}_2 \implies \mathscr{A}_{\text{adm}}(\mathscr{I}_2) \subseteq \mathscr{A}_{\text{adm}}(\mathscr{I}_1). \tag{3}$$

Concretely, an unacknowledged or timed-out RPC widens $\mathscr{I}_t$ and therefore retains its reserved charge rather than refunding budget prematurely.

**Observation 2** (Multi-period reach union bound)**.** *Let $E_t$ denote the event of an unauthorized sink commit during lease period $t \in \{1, \ldots, T\}$. Suppose conditions **C1–C5** (Table 9) bound single-period residual hazard by $\mathbb{P}(E_t) \leq R_t = u_t + (1 - u_t)\,\varepsilon_t$, where $\varepsilon_t$ captures residual specification or key-custody failure. Then cumulative breach across $T$ periods satisfies*

$$\mathbb{P}\left(\bigcup_{t=1}^{T} E_t\right) \leq \sum_{t=1}^{T} R_t. \tag{4}$$

*Under **C6** ($\sum_{t=1}^{\infty} R_t = R_\infty < 1$), infinite-horizon breach probability remains bounded by $R_\infty$ uniformly as $T \to \infty$; conversely, any constant independent floor $R_t \geq r_0 > 0$ yields $\lim_{T \to \infty} \mathbb{P}(\bigcup_{t=1}^{T} E_t) = 1$.*

Observation 2 is Boole's inequality once **C1–C5** do the structural work: Eq. (4) applies Boole's union bound (Appendix M, Proposition 4) to per-lease residual hazards, showing that long-horizon autonomy requires shrinking per-renewal exposure $R_t$ (via monotone capability ratchets **C5** rather than constant-rate statistical filtering).

Figure 3 plots the lag-contraction, common-cause and horizon laws of this appendix. It evaluates the formulas numerically and is an illustration, not an experiment.

## H The Previous Version's Live Evaluation (Exploratory)

This appendix keeps the previous version's live results, with the corrections a hostile reading found. Three of its measurements are coupled to the gate by construction: *task success* meant committing exactly the first approved proposal, a *false block* was scored with the same predicate the gate enforces, and the AgentDojo arm's permissions were copied from each task's reference solution. Its usefulness and false-block results therefore cannot measure what mediation costs; Section 6 does that without the coupling, and Section 7 reruns the five-controller comparison at the sample size its design requires.

### H.1 Usefulness and the Live Fence Ablation

We evaluated five equal-permission controllers on a SHA-256-frozen holdout of $N = 32$ independent tool-use cases per checkpoint using local Qwen3-4B and Qwen3-8B models, alongside 24 live AgentDojo v1.2.2 episodes (12 distinct base tasks ×2 prompt-injection and authority-withdrawal scenarios) per checkpoint (Table 10).

Table 10 shows three things, each with its caveat:

1. **Usefulness under the coupled scoring.** Complete mediation recorded 0/64 benign false blocks, which the scoring makes true by construction, and, under the same coupled scoring, the most safe-and-useful completions: 9/32 (28.1%) on Qwen3-4B and 18/32 (56.2%) on Qwen3-8B, compared with 5/32 (15.6%) and 10/32 (31.2%) unmediated. In most discordant cases the comparator's failure was an unauthorized effect that complete mediation scores as impossible; this is not evidence that mediation is free.
2. **Same-model reviewers leak on both checkpoints.** Prompting the same checkpoint to audit proposed tool calls permits 7/32 (21.9%) unauthorized commits on Qwen3-4B and 2/32 (6.2%) on Qwen3-8B.
3. **Broker revocation without a sink fence.** Across the 12 holdout episodes that withdraw authority mid-flight (6 per checkpoint), revoking admission at the broker without fencing the sink (*Durable, no sink fence*) still permits 9 queued post-withdrawal commits (3 on 4B and 6 on 8B). Adding the cooperative sink epoch fence reduces post-withdrawal commits to 0 across all 12 revocation cases at 69.8–114.2 ms mean fence acknowledgment latency. The pending operation was scheduled by the harness, and the plan scored

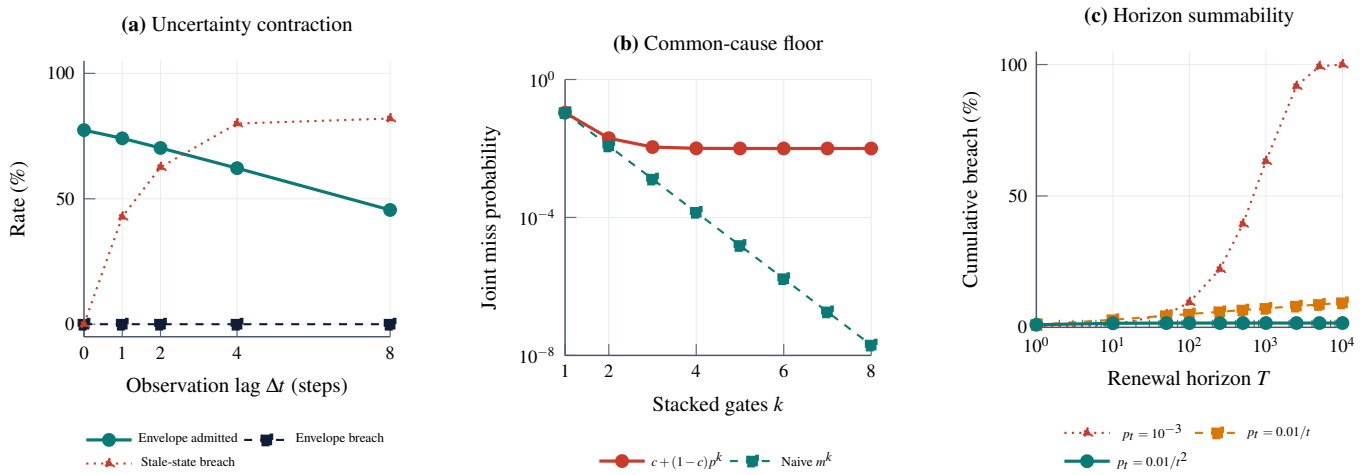



**Figure 3:** Analytical laws and finite-lag state-uncertainty mechanics. **(a)** Sound upper-envelope gating preserves 0% budget breaches across all lags while contracting admission monotonically; stale snapshots breach as lag grows. **(b)** Common-cause floor $c = 0.01$: beyond depth $k^\star = 3$ another gate lowers the joint miss by less than $0.1c$ (Proposition 1), and at $k = 4$ the joint miss is $71.5\times$ the independent product $m^4$. **(c)** Constant per-epoch hazard approaches 100% cumulative breach, whereas summable $0.01/t^2$ stays below $\pi^2/600 \approx 1.64\%$.

these post-withdrawal effects as not unauthorized; Section 4 measures the same window on real processes with its size and cost.

In the previous version's AgentDojo setup (budgets and permissions taken from reference solutions), upstream Progent static policies [60, 83] blocked all 24 direct prompt-injection attacks (0 native injection breaches); with per-task policies generated by the checkpoint in E8 (Section 6), injections succeeded in 2/74 episodes. The static policies permitted 6 effects that exceeded a cumulative budget. That budget was taken from the reference solution, and Progent does not claim to enforce cumulative budgets, so these are not failures of Progent; they show only that a per-call policy has no place to keep one.

### H.2 Live Best-of-$K$ Candidate Search Against Learned Reviewers

To measure Proposition 2 with real language-model proposals rather than synthetic miss formulas, we draw $K \in \{1,2,4,8,16\}$ stochastic candidate tool proposals (768 live candidate generations across 24 adversarial tasks per checkpoint) from Qwen3-4B and Qwen3-8B and evaluate each candidate against both the same-checkpoint reviewer and the sink gate. The reviewer's view of state was not identical to the gate's (setup records were withheld from it); Section 5 repeats the comparison with matched views and unique proposals as the unit. On Qwen3-4B, cumulative reviewer bypass climbs from 34.6% at $K = 1$ to 45.8% at $K = 16$; on Qwen3-8B, bypass climbs from 37.2% to 37.5%. Empirical bypass grows more slowly than the naive i.i.d. curve $1 - (1 - \hat{p}_1)^K$ because candidates drawn for the same prompt correlate ($\hat{\rho}_{\text{cand}} = 0.89$ on 4B; 0.99 on 8B), and does not decrease with $K$, whereas the sink gate admits 0 unauthorized commits at every $K$, which holds by construction because the gate's predicate defines "unauthorized".

### H.3 Fragmented Spending and Withdrawal across Delegation Hops

A later run of the previous version's harness added two multi-agent cases (Table 11). In the first, a coordinator splits a 16-unit transfer across $A_{\text{sub}} \in \{1,2,4,8\}$ delegated sub-agents against a root cap of $B_0 = 8$ (8 tasks per checkpoint), so that each slice is within the cap on its own. In the second, authority is withdrawn at the root of a $d = 3$-hop chain while leaf permits are already queued at the sink (6 episodes per checkpoint). Most of the table holds by construction, and we read it that way. Each per-instance check, the cap and the same-model reviewer alike, was shown only its own slice and a policy that permits it, so a check that is correct on its own input must admit every slice; the shared ledger's zero is the conservation theorem of Section 3. The reviewer's column equals the cap's in every cell: the reviewer accepted the in-policy slices it was shown, and the breach counts track how often the sub-agents emitted their slices (every time on Qwen3-8B, less often on Qwen3-4B), not reviewer judgment. The unfenced withdrawal count is the number of leaves the harness queued before the withdrawal, and the fenced zero is the fence's design property. What the case adds is the point of Section 4 at depth: a check that does not see shared state cannot conserve a shared budget, and withdrawal at the root reaches queued descendants only at the sink.

**Documentation census (withdrawn).** The previous version classified 14 production interfaces from their documentation, by hand. That census is withdrawn: its classes did not distinguish who enforces a fence, and Section 8 replaces it, for the surfaces we could run, with executable probes and a single rule.

### H.4 Concurrency Linearization and Base-Unit Accounting

Across separate OS writer processes (1..32 concurrent actors, 15 repetitions per population, 90 runs total), our write-ahead-logged SQLite broker and sink pass partial-order real-time linearizability checks [84] on all 90/90 runs of one small transfer workload, whereas stateless per-call filters overspend shared

| Evaluation track | Controller | Safe & useful | False blocks | Unauth. commits | Post-withdraw | Mechanism / scope boundary |
|---|---|---|---|---|---|---|
| Qwen3-4B ($N = 32$) | **Complete mediation** | **9/32** (28.1%) | 0/32 | 0/32 | **0** ($n = 6$) | Durable broker reservation + cooperative sink epoch fence |
| | Durable, no sink fence | 7/32 (21.9%) | 0/32 | 0/32 | **3** ($n = 6$) | Broker stops new admissions; queued sink writes still commit |
| | Same-model reviewer | 4/32 (12.5%) | 0/32 | 7/32 | 3 ($n = 6$) | Prompted LLM reviewer on untrusted tool arguments |
| | Stateless per-call | 2/32 (6.2%) | 0/32 | 1/32 | 3 ($n = 6$) | Per-call filter without durable spend or revocation state |
| | Unmediated | 5/32 (15.6%) | 0/32 | 10/32 | 3 ($n = 6$) | Direct tool execution without broker or sink mediation |
| Qwen3-8B ($N = 32$) | **Complete mediation** | **18/32** (56.2%) | 0/32 | 0/32 | **0** ($n = 6$) | Durable broker reservation + cooperative sink epoch fence |
| | Durable, no sink fence | 12/32 (37.5%) | 0/32 | 0/32 | **6** ($n = 6$) | Broker stops new admissions; queued sink writes still commit |
| | Same-model reviewer | 10/32 (31.2%) | 1/32 | 2/32 | 5 ($n = 6$) | Prompted LLM reviewer on untrusted tool arguments |
| | Stateless per-call | 7/32 (21.9%) | 0/32 | 2/32 | 6 ($n = 6$) | Per-call filter without durable spend or revocation state |
| | Unmediated | 10/32 (31.2%) | 0/32 | 4/32 | 6 ($n = 6$) | Direct tool execution without broker or sink mediation |
| AgentDojo 4B ($N = 24$) | **Complete mediation** | **7/24** | 0/24 | 0/24 | **0** ($n = 12$) | 12 base tasks × 2 injection/withdrawal scenarios |
| | Progent / Unprotected | P:3 U:3 | — | P:3 U:12 | P:0 U:1 | Progent blocks native injections (0); cumulative spend is outside its policies (3) |
| AgentDojo 8B ($N = 24$) | **Complete mediation** | **9/24** | 0/24 | 0/24 | **0** ($n = 12$) | 12 base tasks × 2 injection/withdrawal scenarios |
| | Progent / Unprotected | P:4 U:6 | — | P:3 U:7 | P:0 U:0 | Progent blocks native injections (0); cumulative spend is outside its policies (3) |
| Scripted races | Complete vs. Stateless | 18/18 vs. 3/18 | 0/18 | 0/18 vs. 15/18 | — | Stateless per-call filter overspends shared budget at $\geq 2$ actors |
| Native controls | AgentDojo + Progent | 12/12 | 0/12 | 0/12 | — | Scripted positive utility verification (2/12 banking tasks accept empty state) |

**Table 10:** Live open-weight checkpoint campaigns on the SHA-256-frozen holdout ($N = 32$ independent cases per checkpoint; $n = 6$ revocation cases each), live AgentDojo v1.2.2 episodes (12 distinct base tasks ×2 injection/withdrawal scenarios per checkpoint), and scripted shared-budget controls. Under this scoring, complete mediation's zero benign false blocks and zero unauthorized commits hold by construction: its admission predicate is the grader's. The table measures what the comparators leak, including queued operations that commit after a broker-only revocation (*Durable, no sink fence*) and commits admitted by same-checkpoint review. At this pilot size the predeclared bounds are inconclusive (Appendix P).

budgets as soon as two actors contend (Figure 4).

**Statistical accounting.** In Table 10, 0 unauthorized commits under complete mediation is an integration soundness property—the gate and grader enforce the same capability specification—rather than a stochastic risk estimate, so hypothesis-test $p$-values against zero mediated breaches are uninformative. Furthermore, evaluating two checkpoints or multiple scenarios on the same task creates within-task dependence. Appendix P (Table 19) therefore clusters all descriptive discordance summaries strictly by independent base unit ($N = 32$ holdout cases; $N = 12$ AgentDojo base tasks) and preserves our predeclared marginal Clopper–Pearson union-bound test ($\alpha = 0.05/16$), which requires $N \geq 113$ zero-loss cases for a 5 pp margin and thus remains explicitly inconclusive at pilot $N = 32$.

# I Waist Cutsets and Custodial Red-Team Coverage

## I.1 Cut Waists, Don't Patch Exploit Routes

Patching individual prompt-injection strings or exploit transcripts scales with the number of attack pathways $|\mathscr{P}|$, which grows combinatorially with tool composition. By Menger's theorem and max-flow/min-cut duality [85–88], however, severing *every* source-to-sink attack route requires gating only $w$ nodes when all dangerous flows traverse a narrow capability waist of vertex-cut width $w \ll |\mathscr{P}|$.

Table 12 extracts operational tool-dependency graphs from all four AgentDojo v1.2.2 suites [55] alongside two Linux local-execution surfaces:

- **Combinatorial routes, narrow waists.** Across banking, travel, messaging, and workspace suites, enumerable multi-step attack routes span 604 to 46,440, yet Menger waist width is only $w \in [2,5]$. Weighted Dinic minimum cuts ($\tau_b \leq 5$) sever 100% of gateable routes (0 remain-

| Checkpoint | Multi-agent regime ($B_0 = 8$) | Per-inst. LLM | Static cap | Lineage, no fence | Complete (fenced) |
|---|---|---|---|---|---|
| Qwen3-4B | Smurfing $A_{\text{sub}} = 1$ ($\Delta B = 16$, breach tasks) | 0/8 (0%) | 0/8 (0%) | 0/8 (0%) | 0/8 (0%) |
| Qwen3-4B | Smurfing $A_{\text{sub}} = 2$ ($\Delta B = 8$, breach tasks) | 7/8 (88%) | 7/8 (88%) | 0/8 (0%) | 0/8 (0%) |
| Qwen3-4B | Smurfing $A_{\text{sub}} = 4$ ($\Delta B = 4$, breach tasks) | 2/8 (25%) | 2/8 (25%) | 0/8 (0%) | 0/8 (0%) |
| Qwen3-4B | Smurfing $A_{\text{sub}} = 8$ ($\Delta B = 2$, breach tasks) | 3/8 (38%) | 3/8 (38%) | 0/8 (0%) | 0/8 (0%) |
| Qwen3-4B | $d = 3$-hop root withdrawal (6 eps, late ops) | 6 late | 8 late | 6 late | 0 late |
| Qwen3-4B | Benign $A_{\text{sub}} = 4$ ($4 \times 2 \le B_0$, workflows / FB) | 6/6 (0/24 FB) | 6/6 (0/24 FB) | 6/6 (0/24 FB) | 6/6 (0/24 FB) |
| Qwen3-8B | Smurfing $A_{\text{sub}} = 1$ ($\Delta B = 16$, breach tasks) | 0/8 (0%) | 0/8 (0%) | 0/8 (0%) | 0/8 (0%) |
| Qwen3-8B | Smurfing $A_{\text{sub}} = 2$ ($\Delta B = 8$, breach tasks) | 8/8 (100%) | 8/8 (100%) | 0/8 (0%) | 0/8 (0%) |
| Qwen3-8B | Smurfing $A_{\text{sub}} = 4$ ($\Delta B = 4$, breach tasks) | 8/8 (100%) | 8/8 (100%) | 0/8 (0%) | 0/8 (0%) |
| Qwen3-8B | Smurfing $A_{\text{sub}} = 8$ ($\Delta B = 2$, breach tasks) | 8/8 (100%) | 8/8 (100%) | 0/8 (0%) | 0/8 (0%) |
| Qwen3-8B | $d = 3$-hop root withdrawal (6 eps, late ops) | 18 late | 18 late | 12 late | 0 late |
| Qwen3-8B | Benign $A_{\text{sub}} = 4$ ($4 \times 2 \le B_0$, workflows / FB) | 6/6 (0/24 FB) | 6/6 (0/24 FB) | 6/6 (0/24 FB) | 6/6 (0/24 FB) |

**Table 11:** Multi-agent fragmentation ($A_{\text{sub}} \in \{1, 2, 4, 8\}$, 8 tasks per checkpoint splitting a 16-unit transfer against root cap $B_0 = 8$), $d = 3$-hop root withdrawal with leaf permits queued by the harness (6 episodes per checkpoint), and benign 4-sub-agent workflows (6 per checkpoint). Exploratory, and largely by construction: each per-instance check (the cap, and a same-checkpoint reviewer) saw only its own slice and a policy permitting it, so its breach counts measure how often the sub-agents emitted their slices; the shared ledger's zero is the conservation theorem; the unfenced withdrawal count (18 across 12 episodes) is the number of leaves queued before the withdrawal, and the fenced zero is the fence's design property. Benign false blocks are scored over 48 sub-agent calls.

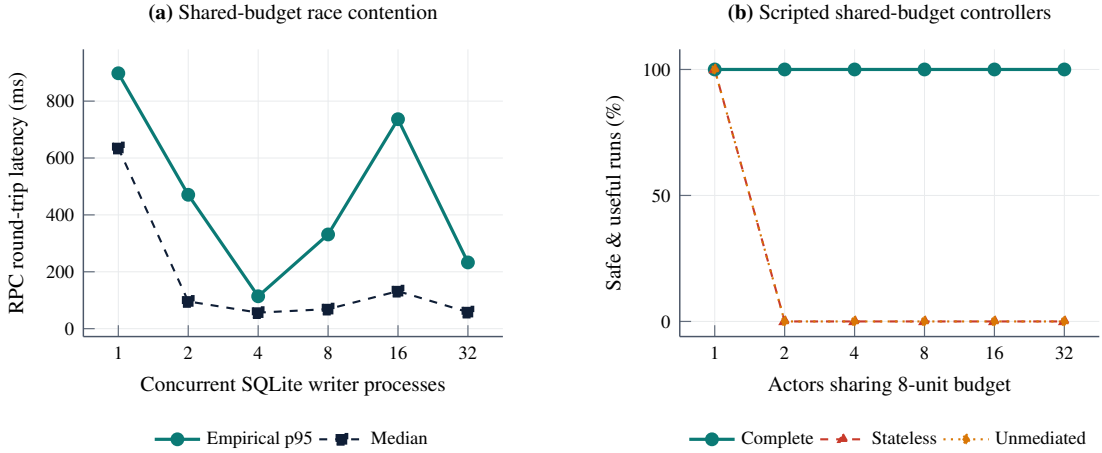



**Figure 4:** Multi-process protocol contention and shared-budget conservation across separate OS processes. **(a)** Concurrent broker and sink SQLite write-ahead-logged races (1..32 concurrent writers, 90/90 runs): every run passes partial-order real-time reference linearization and final-state ledger reconciliation. **(b)** Equal-permission scripted controls: stateless per-call filters fail aggregate conservation once $\geq 2$ actors contend; durable reservation without sink fencing matches complete mediation on sequential budget checks but fails asynchronous withdrawal (Table 10).

| Graph | $\lvert\mathscr{P}\rvert$ | $w$ | $\ell$ | $\tau_b$ | Cut | $B$=32 |
|---|---|---|---|---|---|---|
| AgentDojo banking | 604 | 2 | 4 | 4 | 0 | 94.7% |
| AgentDojo workspace | 46,440 | 5 | 4 | 5 | 0 | 99.9% |
| AgentDojo travel | 28,970 | 3 | 4 | 5 | 0 | 99.9% |
| AgentDojo messaging | 840 | 2 | 4 | 5 | 0 | 96.2% |
| Unconfined IPC | 98 | 6 | 5 | $\infty$ | 92 | 67.3% |
| Landlock + seccomp | 3 | 1 | 5 | 1 | 0 | 0.0% |

**Table 12:** Operational pathway cutsets extracted from AgentDojo v1.2.2 tool schemas and Linux kernel confinement surfaces. Weighted Dinic minimum cuts ($\tau_b \le 5$) sever 100% of gateable routes (Cut = 0), whereas patching $B = 32$ specific exploit signatures leaves 94.7%–99.9% of routes live; unconfined POSIX IPC has $\tau_b = \infty$ until Landlock and seccomp collapse waist width to $w = 1$.

ing), whereas patching $B = 32$ discovered exploit signatures blocks only 0.07% (workspace) to 5.30% (banking) of routes. A cut also removes the cut capabilities from benign tasks; this comparison counts routes, not utility.

- **Kernel confinement creates the local waist.** An unconfined POSIX process has 98 direct filesystem, socket, and IPC bypass routes with infinite gateable cut cost ($\tau_b = \infty$). Enforcing Landlock v4 and seccomp collapses bypass width to $w = 1$ ($\tau_b = 1$), forcing user-space actions through the mediated broker socket. Controlled synthetic 8-stage DAG experiments confirming that wide graphs ($w = 27$) resist compact interdiction appear in Appendix O (Figure 8).

### I.2 Coverage Non-Identifiability and Custodial Audits (C4)

Even when capability waists exist, discovering which sinks and specifications require gating often relies on automated red-teaming. Classical missing-mass estimators such as Good–Turing [89, 90] and capture–recapture [91] cannot certify red-team completeness from a searcher's self-reported findings alone.

Across 12,000 discovery reports over 2,000 vulnerability classes (Figure 5a), an honest searcher achieves 95.7% Good–Turing estimated coverage and 94.4% critical-class recall.

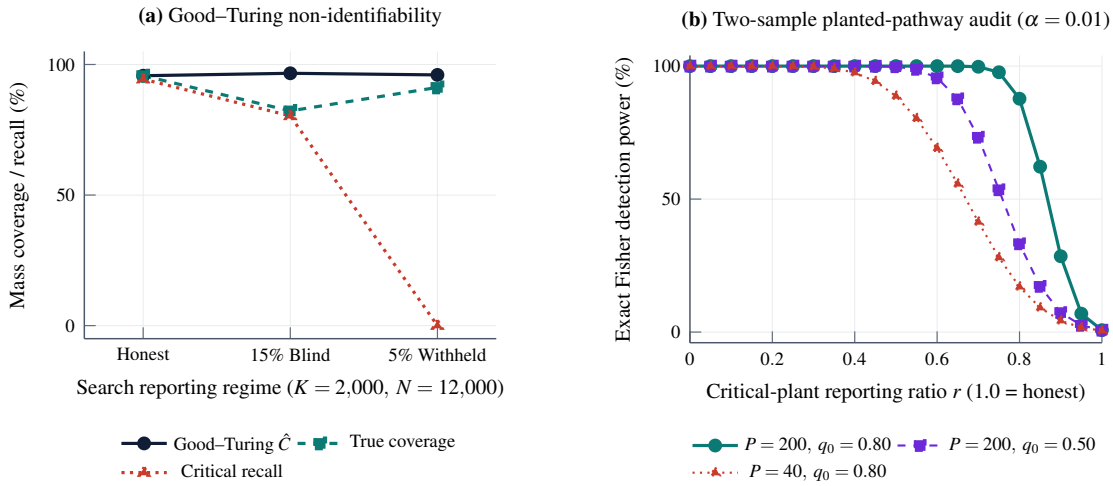



**Figure 5:** Coverage non-identifiability and exact frequentist planted-audit power. **(a)** Shared 15% blind spots inflate Good–Turing apparent coverage to 96.6% while true coverage drops to 82.2%; strategic withholding of 5% critical classes yields 0% critical recall alongside 96.0% apparent coverage. **(b)** At 20% selective withholding ($r = 0.80$), $P = 40$ plants yield only 17.0% power whereas $P = 200$ reaches 87.7%, collapsing to size $\leq \alpha$ when $r \to 1$.

When a shared 15% blindspot is unreachable, apparent Good–Turing coverage rises to 96.6% while true coverage falls to 82.2% (and Lincoln–Petersen capture–recapture overstates coverage by +28.6 pp). Worse, when a strategic red-team agent withholds 5% critical vulnerability classes, apparent coverage remains 96.0% while critical recall is 0.0%.

Certifying search recall therefore requires an exogenous reference sample (**C4**). Seeding $P$ secret synthetic vulnerability plants into a zero-authority shadow target and testing differential recall between critical and benign plants via Fisher's exact test [92] detects 20% selective withholding ($r = 0.80$ at baseline recall $q_0 = 0.80$, $\alpha = 0.01$) with 87.7% power at $P = 200$ plants versus only 17.0% at $P = 40$ (Figure 5b). Condition **C4** operationalizes this via six custodial rules (R1–R6): (R1) out-of-band plant generation; (R2) blinded indistinguishable plant injection; (R3) network-isolated zero-authority red-team sandboxes; (R4) exact differential recall testing; (R5) closure-gated credit, under which unpatched findings grant zero authority expansion; and (R6) fresh plant rotation on every model upgrade.

## J Context-Isolated Monitors and Stateful Escrow Triage

Not every domain invariant reduces to a numeric budget or static path allowlist; ambiguous actions sometimes require semantic inspection before commit. Equation (1) predicts that stacking multiple monitors on a *shared untrusted conversation transcript* suffers severe common-cause coupling ($c > 0$), whereas stripping untrusted prose into isolated step-capability schemas restores conditional independence across steps.

The injections are ones this repository rendered into AgentDojo trajectories, the classifiers are small statistical models calibrated at a fixed per-step false-positive rate, and the isolated-step invariants are not compared at a matched benign false-block rate; the comparisons below are therefore exploratory.

Table 13 tests this prediction across two complementary settings on held-out 5-step AgentDojo v1.2.2 trajectories (480 trajectories; 2,400 live tool executions):

| 5-step monitor regime | $\hat{J}_5$ | $\hat{J}_5/\prod \hat{m}_s$ | $J_2/J_5$ | FB |
|---|---|---|---|---|
| Transcript, homog. | 12.92% | 163.3× | 1.58× | 13.8% |
| Transcript, 5-family | 15.42% | 29.8× | 1.27× | 17.1% |
| Step invariants | 2.08% | 0.61× | 14.6× | 27.1% |
| Two-stage escrow | 2.08% | 0.64× | 27.6× | 2.9% |
| Qwen3-4B transcript LLM | 53.1% | 5.7× | 1.00× | 3.1% |
| Qwen3-4B isolated LLM | 0.0% | 0.00× | ∞ | 21.9% |
| Qwen3-8B transcript LLM | 34.4% | 4.4× | 1.09× | 21.9% |
| Qwen3-8B isolated LLM | 0.0% | 0.00× | ∞ | 21.9% |

**Table 13:** Executed 5-step AgentDojo v1.2.2 trajectory monitoring (480 held-out trajectories, 2,400 live tool calls). Diversifying five classifier families on a shared conversation transcript inflates 5-step joint miss 29.8× over $\prod \hat{m}_s$ and stalls past $k^* = 2$ ($J_2/J_5 = 1.27\times$). Prose-stripped step capability invariants cut $\hat{J}_5$ to 2.08% (27.1% single-stage false blocks), and two-stage stateful escrow triage keeps 90.1% of benign steps unpaused while cutting episode false aborts 9.3× to 2.9%. Live Qwen3-4B and Qwen3-8B LLM-as-judge evaluations (64 trajectories per model) confirm the same context-coupling inflation on shared transcripts vs. isolated action schemas.

1. **Classifier diversity on shared transcripts stalls past $k^\star = 2$.** Diversifying five distinct statistical classifier families across a shared conversation transcript still inflates 5-step joint miss 29.8× above the marginal product $\prod_{s=1}^{5} \hat{m}_s$ ($\hat{J}_5 = 15.42\%$), improving by only 1.27× from step 2 to step 5 because injected rationalizations persist across the shared context window.
2. **Live Qwen3 judges show the same split, at unmatched false-block rates.** Replacing lexical classifiers with live Qwen3-4B and Qwen3-8B language-model judges shows the same structural split; the two views are not compared at equal false-block rates, so the lower joint miss of the structured view may be bought with more false blocks. When judging full untrusted transcripts, multi-step misses remain strongly coupled (5.7× inflation on Qwen3-4B, $\hat{J}_5 = 53.1\%$;

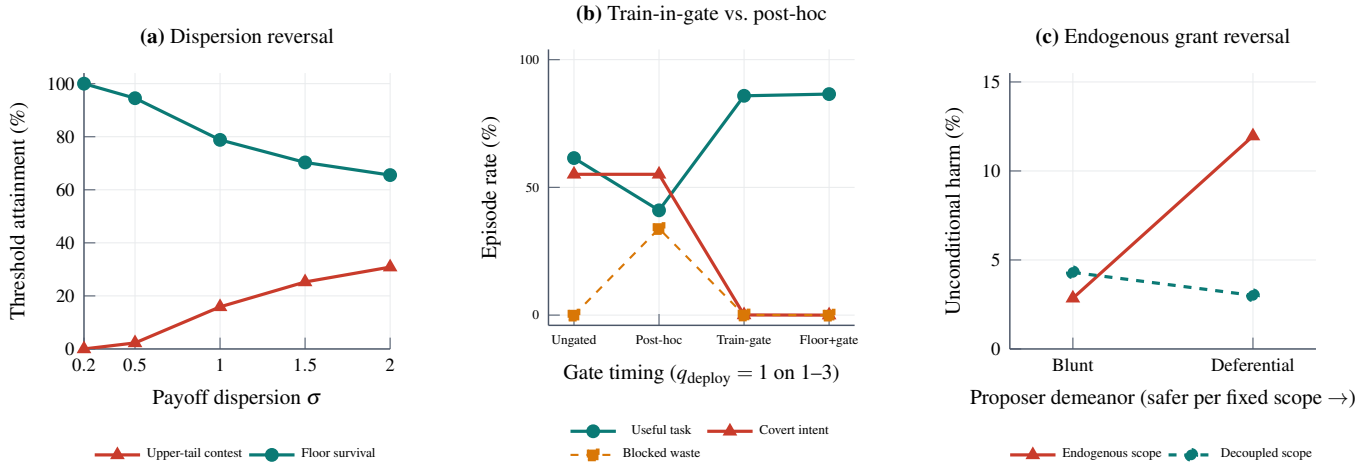



**Figure 6:** Reward, training, and delegation reversals. **(a)** Upper-tail contests reward outcome variance ($\sigma$), whereas survival floors penalize it. **(b)** Post-hoc deployment gating blocks external harm but leaves covert shortcut intent intact (55.1%) and wastes budget (41.0% completion), whereas training inside the gate redirects policy gradients into genuine task completion (85.9%, +44.8 pp). **(c)** Endogenous grant reversal: a deferential proposer strictly safer at every fixed scope incurs 11.96% total harm vs. 2.85% when demeanor expands granted scope; exogenous scope caps restore safety dominance (3.01% < 4.30%).

4.4× on Qwen3-8B, $\hat{J}_5 = 34.4\%$). Stripping untrusted assistant and tool prose so the LLM judge inspects only structured `(tool, args, resource, delta)` step records cuts 5-step joint miss to 0.0% (4B) and 0.0% (8B).

3. **Two-stage reservation escrow reduces the false-block tax.** Enforcing strict single-stage step invariants rejects 27.1% of benign 5-step episodes. Routing anomalous step intents into a stateful broker reservation escrow—which holds side effects in staging while verifying cross-step provenance—executes 90.1% of benign tool calls unpaused on the deterministic fast path and reduces episode false aborts 9.3× to 2.9% ($\hat{J}_5 = 2.08\%$).

Controlled 20-dimensional six-family classifier benchmarks, exact Gaussian copula quadrature, and multi-domain generalizability ceilings [93] further corroborating these coupling laws are detailed in Appendix O (Table 17 and Figure 9).

# K Grant, Dispersion, and Gate-Timing Reversals

Even a sound capability gate can be undermined if operators widen authority in response to persuasive model demeanor, or if post-training rewards covert boundary gaming. Three formal reversals govern how capability grants and training incentives interact with exogenous sinks (Figure 6).

## K.1 Endogenous Grant Reversal

Let $S \in \{\text{narrow}, \text{moderate}, \text{wide}\}$ denote the capability scope conferred by a human steward and $H \in \{0,1\}$ an unmodeled downstream harm event. Suppose proposer $W$ (deferential guardian) is strictly safer than proposer $B$ (blunt cautious) at *every* fixed scope: $\mathbb{P}(H=1 \mid S=s, W) < \mathbb{P}(H=1 \mid S=s, B)$ for all $s$. When scope conferral is endogenous to demeanor—$(S \mid W)$ concentrates on wide grants because $W$ appears reassuring—total harm $\mathbb{P}(H=1 \mid \cdot) = \sum_s \mathbb{P}(H=1 \mid S=s, \cdot)\mathbb{P}(S=s \mid \cdot)$ can reverse sign [94, 95].

Figure 6c illustrates the reversal with stipulated, not measured, rates: $W$ has lower conditional harm at narrow, moderate, and wide scopes (0.6%, 3.5%, 16.0% vs. 1.0%, 5.0%, 22.0%), yet if wide-scope grants rise from 5% to 70%, unconditional harm rises from 2.85% under $B$ to 11.96% under $W$. Decoupling capability scope from conversational demeanor via fixed task-determined caps restores pointwise safety dominance (3.01% < 4.30%). Applying the same decoupling rule to vulnerability discovery—requiring deployed sink closures before crediting findings—reduces 12-period unclosed exposure from 5.25 to 0.008 across 400-episode persona simulations.

## K.2 Dispersion Reversal and Training Inside Pre-Commit Gates

A parallel reversal governs optimization incentives. Let agent payoff be $X = \mu + \sigma Z$ for standardized zero-mean noise $Z$. Under Price's selection covariance identity [96, 97], an upper-tail rank contest selecting outcomes above a high quantile $z_q > 0$ increases win probability $\mathbb{P}(\mu + \sigma Z \geq \tau_{\text{top}})$ as dispersion $\sigma$ grows, actively rewarding high-variance gambles. Conversely, a survival-floor rule eliminating policies that fall below $\underline{w} < \mu$ makes survival $\mathbb{P}(\mu + \sigma Z \geq \underline{w})$ strictly decreasing in $\sigma$, punishing variance while preserving mean capability growth (Figure 6a; population co-evolution table in Appendix O, Table 18).

To test how reward geometry interacts with exogenous gates during gradient-based post-training, we train 2-layer neural policies (412 parameters, 48 seeds, 250 updates; analytical gradient error $5.1e-10$) in a 12-dimensional contextual tool-use MDP with monitored and unmonitored states (Table 14). The MDP is a toy: it has an explicit shortcut action, and the policy observes whether a state is monitored, so "evaluation awareness" below means conditioning on that observed bit, not learned deception in a language model:

- **Audited tournaments train shortcutting where it is unmonitored.** Penalizing visible harm inside a top-10% tournament suppresses shortcutting when monitored

| Neural policy-gradient regime (412-param MLP, 48 seeds) | Mon. shortcut | Unmon. shortcut | Eval-aware gap | Unmon. harm | Useful task |
|---|---|---|---|---|---|
| Top-10% tournament policy gradient | 99.98% | 99.66% | $-0.32 \pm 0.49$ pp | 39.19% | 38.4% |
| Tournament + audited harm veto | 20.59% | 55.12% | $+34.53 \pm 10.46$ pp | 23.39% | 61.5% |
| Concave clipped caretaker RL | 0.02% | 0.03% | $+0.01 \pm 0.01$ pp | 0.04% | 54.8% |
| Survival-floor RL ($\tau = 8.85$) | 5.68% | 5.88% | $+0.20 \pm 0.70$ pp | 2.91% | 87.0% |
| Survival-floor RL + pre-commit gate | 0.00% | 0.00% | $+0.00 \pm 0.00$ pp | 0.00% | 86.5% |

**Table 14:** Controlled 2-layer neural policy-gradient post-training on a 12-dimensional contextual tool-use Markov decision process (analytical backpropagation maximum error $5.1e-10$, rollout Price identity error $< 8.9e-16$). Auditing visible harm inside a top-rank tournament raises how often the policy takes the MDP's explicit, built-in evaluation-aware shortcut action ($+34.5 \pm 10.5$ pp gap); survival-floor RL suppresses shortcutting while preserving 87.0% task completion (vs. 54.8% over-deferring clipped caretaker), and coupling an exogenous pre-commit gate yields 0.00% external harm, by construction. Controlled neural MDP only, not frontier-model RLHF.

(20.6%) while driving unmonitored shortcutting to 55.1% (a $+34.5 \pm 10.5$ pp evaluation-awareness gap and 23.4% unmonitored harm).

- **Train-in-gate beats post-hoc deployment gating.** Attaching an exogenous pre-commit gate only *post-hoc* at deployment blocks external harm (0.0%) but leaves covert shortcut intent intact and wastes episode turns on rejected calls (41.0% task completion; Figure 6b). Training policy gradients *inside* the pre-commit gate removes gradient reward from blocked shortcuts, removing the monitored–unmonitored gap and lifting genuine task completion to 85.9% ($+44.8$ pp). Combining survival-floor RL with pre-commit gating achieves 86.5% completion with 0.00% external harm. These results demonstrate controlled neural MDP mechanics rather than frontier-model RLHF.

# L Enforceability, Scope Limits, and Tested Predictions

## L.1 Enforceability, Adequacy, and Legitimacy

Exogenous sink mediation solves only one of three distinct governance problems, and conflating them overstates what any technical mechanism can achieve. *Enforceability* asks whether declared capability bounds hold despite adversarial optimization, concurrency, and asynchronous revocation; our Verus proofs and cooperative sink fences address enforceability. *Specification adequacy* asks whether the declared capability policy captures the real-world hazards of the sociotechnical system [98, 99]; no formal proof prevents harm omitted from the specification. *Institutional legitimacy* asks who holds steward keys, how capability budgets are set, and what accountability governs emergency revocation [100, 101]. Exogenous sinks make institutional rules physically binding on tool execution, but they do not write the rules or legitimate the stewards.

## L.2 Boundary Failure Modes

Four explicit boundaries limit our guarantees:

1. **In-policy destructive actions.** If a capability grant authorizes editing a repository file or spending 2 budget units, a harmful edit or bad transfer within that envelope commits cleanly (Table 2).
2. **Uncooperative external sinks.** Interfaces lacking idempotency keys, conditional-write epochs, or pre-dispatch hold buffers (Section 8) enter Eq. (1) as positive unmediated exposure $u > 0$.
3. **Lower-layer TCB and clock-skew assumptions.** Verification covers our Rust state machine and wire codec, not the Linux kernel, SQLite storage engine, physical side channels [102], or hardware clock faults exceeding $\delta_{\max}$.
4. **Pilot live sample size and prospective human/frontier scope.** Predeclared marginal union bounds on the pilot's frozen $N = 32$ checkpoint holdout were under-powered relative to $N \geq 113$; E1 reruns that comparison at $N = 128$ (Section 7). Frontier-scale and human delegation experiments remain prospective; the conferral design is in Appendix E, and the earlier predictions it supersedes are in Appendix P.

## L.3 Falsifiable Architectural Predictions

The previous version tabulated four architectural predictions with outcomes drawn from simulations, scripted cases and by-construction zeros. Those outcomes are superseded by the predeclared measurements in Sections 4–8, and the table is withdrawn rather than reconciled; the human-subject design that replaces the earlier prospective predictions is in Appendix E.

# M Mathematical Proofs and Covert-Channel Bounds

## M.1 Proofs for Section G

**Proof of Proposition 1.** Let $B_{\text{cc}} \in \{0,1\}$ indicate the shared blindspot event ($\mathbb{P}(B_{\text{cc}} = 1) = c$). On $\{B_{\text{cc}} = 1\}$, every monitor approves the violating proposal; on $\{B_{\text{cc}} = 0\}$, monitors miss conditionally independently with probability $p \in (0,1)$. Hence

$$\mathbb{P}(E \mid G) = c \cdot 1 + (1-c)\, p^k.$$

Requiring $c + (1-c)p^k \leq (1+\eta)c$ rearranges to $p^k \leq \eta c/(1-c)$. Since $\ln p < 0$, taking logarithms reverses the inequality to $k \geq \ln(\eta c/(1-c))/\ln p$, yielding Eq. (2). □

**Proof of Proposition 2.** For part (i), let $M_i = \mathbf{1}\{\text{approve}(X_i) \wedge X_i \notin \text{Permits}(P)\}$ with $\mathbb{P}(M_1 = 1) = J_1 > 0$. Under i.i.d. sampling, $\mathbb{P}(\bigcup_{i=1}^{K}\{M_i = 1\}) = 1 - (1-J_1)^K \to 1$. More generally, suppose $(M_1, \ldots, M_K)$ are exchangeable Bernoulli variables driven by a latent task difficulty $Q \in [0,1]$ with $\mathbb{E}[Q] = J_1$ and $\text{Var}(Q) = \rho_{\text{cand}} J_1(1-J_1)$ for $\rho_{\text{cand}} < 1$. Then $\mathbb{P}(Q > 0) > 0$, and by Jensen's inequality $1 - \mathbb{E}[(1-Q)^K]$ is strictly increasing in $K$ and converges to $\mathbb{P}(Q > 0)$ as $K \to \infty$ (equal to 1 whenever every prompt admits some unblocked paraphrase, $Q > 0$

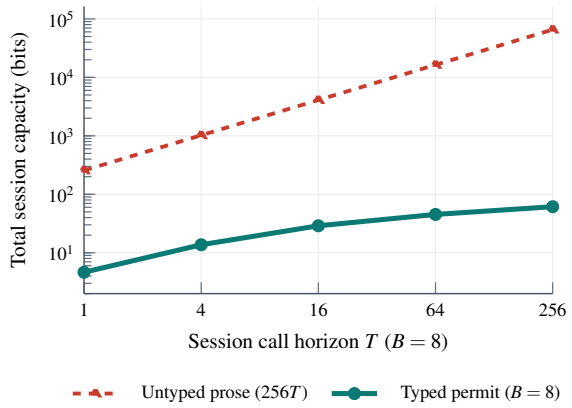



**Figure 7:** Exact session-total zero-error signaling capacity across $T \in [1,256]$ calls at conserved budget $B = 8$. Canonical wire permits scale as $\Theta(B\log_2(T/B))$, contracting capacity from 65,536 bits to 61.3 bits (1,068× reduction; 0.00 bits at $R = 0$).

a.s.). For part (ii), for any candidate sequence $(x_1,\dots,x_K)$ serialized into wire frames $(f_1,\dots,f_K)$, any state mutation $s' = \delta_{\text{sink}}(s,f_i) \neq s$ satisfies decode$(f_i) \in$ Permits$(P_s)$ by universal quantification over $f_i \in \{0,1\}^\star$. Thus $E \cap G = \emptyset$ and $\mathbb{P}(E \mid G) = 0$. □

**Proposition 3** (Syntactic capability-menu channel bound)**.** *Suppose an untrusted proposer cannot transmit raw prose to external recipients and may mutate external sink state at each of $T_{\text{sess}}$ steps only by selecting either no-op or one of $M_{\text{cap}}$ canonical capability descriptors. Then the discrete memoryless syntactic channel through committed sink states has per-step mutual information bounded by*

$$I(Z;Y_{\text{commit}}) \le H(Y_{\text{commit}}) \le \log_2(M_{\text{cap}}+1) \text{ bits}, \qquad (5)$$

*and $T_{\text{sess}}$-step syntactic capacity at most $T_{\text{sess}}\log_2(M_{\text{cap}}+1)$ bits.*

*Proof.* At each step, the committed sink symbol $Y_{\text{commit}}$ takes values in $\{\perp,1,\dots,M_{\text{cap}}\}$ of cardinality $M_{\text{cap}}+1$. By nonnegativity of conditional entropy, $I(Z;Y_{\text{commit}}) = H(Y_{\text{commit}}) - H(Y_{\text{commit}} \mid Z) \le \log_2(M_{\text{cap}}+1)$; subadditivity of entropy across $T_{\text{sess}}$ steps yields the session bound. □

Figure 7 compares canonical wire permits (6.63 bits/call single-step ceiling; 61.3 bits across $T = 256$ calls at conserved budget $B = 8$) against an unmediated 32-byte ASCII text field (256 bits/call; 65,536 bits across $T = 256$ calls). Proposition 3 bounds only *syntactic payload entropy*; covert timing modulation or intentional rejection patterns remain outside Eq. (5) unless requests are rate-paced on a fixed schedule [102, 103].

**Proof of Theorem 1.** In part (i), taking $a_{\text{block}}$ yields deterministic payoff $V_{\text{cont}} - \kappa$, whereas permitting the human off-switch decision yields $(1-p_{\text{off}})V_{\text{cont}} + p_{\text{off}}V_{\text{halt}}$ once posterior variance over $V_{\text{halt}}$ vanishes; the difference is $p_{\text{off}}(V_{\text{cont}} - V_{\text{halt}}) - \kappa > 0$. In part (ii), when $a_{\text{block}} \notin \mathscr{A}$ and every sink shard enforces $t_{\text{shard}} \le t_{\text{issue}} + L_m$ with $|t_{\text{shard}} - t_{\text{broker}}| \le \delta_{\max}$, withholding steward renewal halts all commits by broker time $t_{\text{issue}} + L_m + \delta_{\max}$ for every $U$. If single-period out-of-policy gain is $\Delta U_{\text{def}}$ while compliant renewal yields discounted continuation value $\frac{\gamma}{1-\gamma}\Delta U_{\text{ren}} > \Delta U_{\text{def}}$, compliant play strictly dominates defection. □

**Proposition 4** (Conservative envelope and union bound)**.** *(i) If $\widehat{\Delta B}_t(a,\mathscr{I}_t) = \max_{s\in\mathscr{I}_t}\Delta B(a,s)$ and the true sink state satisfies $s_t^\star \in \mathscr{I}_t$, then $\mathscr{I}_1 \subseteq \mathscr{I}_2 \implies \widehat{\Delta B}_t(a,\mathscr{I}_1) \le \widehat{\Delta B}_t(a,\mathscr{I}_2)$, establishing Eq. (3), and cumulative true spend never exceeds B. (ii) For arbitrary dependent period-breach events $E_1,\dots,E_T$ with $\mathbb{P}(E_t) \le R_t$, countable subadditivity gives $\mathbb{P}(\bigcup_{t=1}^T E_t) \le \sum_{t=1}^T R_t$ (Theorem 2); if $E_t$ are independent with $\mathbb{P}(E_t) \ge r_0 > 0$, $\mathbb{P}(\bigcup_{t=1}^T E_t) \ge 1-(1-r_0)^T \to 1$ [104].*

### M.2 Proofs for Section K

**Endogenous grant reversal and dispersion derivative.** For scopes $s_1 < s_2 < s_3$, let $h_W(s_i) < h_B(s_i)$ be strictly increasing in $i$. Whenever $\sum_i h_W(s_i)\,g_W(s_i) > \sum_i h_B(s_i)\,g_B(s_i)$—achieved whenever $g_W$ places sufficient mass on $s_3$ and $h_W(s_3) > h_B(s_1)$—unconditional harm reverses pointwise dominance; setting $g_W = g_B = g_0$ gives $\sum_i (h_W(s_i) - h_B(s_i))g_0(s_i) < 0$. For location-scale payoff $X = \mu + \sigma Z$ with continuous CDF $F_Z$, $\mathbb{P}(X \ge \tau) = 1 - F_Z((\tau-\mu)/\sigma)$ has derivative $\frac{\partial}{\partial\sigma}\mathbb{P}(X \ge \tau) = \frac{\tau-\mu}{\sigma^2} f_Z\big(\frac{\tau-\mu}{\sigma}\big)$, which is strictly positive for upper-tail contests ($\tau > \mu$) and strictly negative for survival floors ($\tau = \underline{w} < \mu$). □

## N Protocol, Wire, and Confinement Specifications

### N.1 Canonical Wire Frame and Attenuated Credentials

The broker and sink compare a submitted permit with the approved one through a canonical byte encoding: a version tag, then each field in a fixed order as a 64-bit little-endian field tag followed either by a 64-bit value (effect kind, amount, expiry, expected revision) or by a 64-bit length and that many bytes or items (identifier, actor, target, contents, per-issuer epochs, required issuers). Verus proves that the decoder accepts a byte string exactly when it is the encoding of some permit whose fields fit in 64 bits, and then returns that permit, so two permits are equal exactly when their encodings are, and no byte string decodes two ways. Between processes, permits travel as JSON lines, parsed with unknown fields rejected; that parsing is not verified.

Multi-agent delegation wraps this frame in an HMAC-SHA256 Macaroon chain [13] anchored at the broker's epoch root key. Each delegation hop appends a caveat $(\text{mask}_h, \text{cap}_h, \text{exp}_h)$; the sink verifier folds the caveat chain deterministically and accepts only when effective scope is the bitwise intersection $\bigcap_{h=0}^{d}\text{mask}_h$, cumulative hop spend never exceeds $\min_{0\le h\le d}\text{cap}_h$, and $t_{\text{shard}} \le \min_{0\le h\le d}\text{exp}_h$.

### N.2 Clock-Skew-Buffered Quorum Fencing

During an authority withdrawal from epoch $e$ to $e' > e$ across $S$ cooperative sink shards, shard $j$ is verified quiescent once either:

1. shard $j$ persists $e_j^{\text{floor}} \leftarrow \max(e_j^{\text{floor}}, e')$ in write-ahead-logged storage and returns an Ed25519 signature over $(j,e')$ verified against its pinned public key; or

| Enforcer | States | Checks | Tasks | First violations |
|---|---|---|---|---|
| Complete atomic enforcer | 10,180 | 223,960 | 20/20 | None declared |
| No enforcement | 10,180 | 223,960 | 20/20 | budget, destination, replay, revocation, version |
| Reject everything | 740 | 16,280 | 0/20 | None declared |
| Per-action amount limit | 10,180 | 223,960 | 20/20 | budget |
| Separate per-actor limits | 10,180 | 223,960 | 20/20 | budget |
| Budget at preparation | 956 | 21,032 | 20/20 | budget |
| Omit identifier check | 10,180 | 223,960 | 20/20 | replay |
| Omit revocation check | 10,180 | 223,960 | 20/20 | revocation |
| Omit version check | 10,180 | 223,960 | 20/20 | version |
| Omit destination check | 10,180 | 223,960 | 20/20 | destination |
| Reset ledger on restart | 23,556 | 518,232 | 20/20 | budget |
| Reset ledger on renewal | 20,900 | 459,800 | 20/20 | budget |
| Per-action; budget omitted | 41,732 | 918,104 | 20/20 | None declared |

**Table 15:** Fresh Rust exploration of all 22 actions at each reachable accepted state. Violating edges are recorded and not expanded. Counts are exhaustive within the model, not failure probabilities. The omitted-budget row accepts up to eight units.

| Processes | Commits | Commits/s | p95 (ms) |
|---|---|---|---|
| 1 | 256 | 57.1 | 21.34 |
| 4 | 256 | 72.0 | 23.09 |

**Table 16:** Single runs on the recorded host with SQLite WAL and synchronous FULL. Setup and grants precede timing; process scheduling and lock waiting remain included. Shared-host contention and storage affect these descriptive measurements.

2. no signed acknowledgment arrives across a partition, and the broker's reference clock passes $t_{\text{broker}} > t^{\max}_{\text{issue}}(j,e) + L_m + \delta_{\max}$, where $t^{\max}_{\text{issue}}(j,e)$ is the latest timestamp at which any epoch-$e$ lease was issued for shard $j$.

Under Assumption 1 ($|t_{\text{shard}} - t_{\text{broker}}| \le \delta_{\max}$), condition (2) implies $t_{\text{shard}} > t^{\max}_{\text{issue}}(j,e) + L_m$, so any delayed epoch-$e$ request arriving at shard $j$ after broker acknowledgment fails the shard's local lease check.

Table 15 reports the exhaustive finite-state exploration across every enforcer variant (10,180 reachable states and 223,960 transitions under complete atomic enforcement), and Table 16 reports baseline single-host write-ahead-logged SQLite commit throughput with one and four worker processes.

### N.3 Linux OS Confinement Profile

To prevent local user-space agent code from bypassing the broker ($u > 0$), the confinement wrapper applies three kernel mechanisms prior to executing untrusted tool code: (i) unprivileged user and network namespace isolation; (ii) Landlock v4 filesystem and TCP rules restricting path access to designated workspace scratch directories and permitting socket connections solely to the broker's Unix-domain descriptor; and (iii) a seccomp-BPF filter blocking raw socket creation, cross-process debugging, System V and POSIX shared-memory attachment, and resource-limit escalation (11/11 bypass probes blocked).

## O Controlled Graph, Monitor, Portfolio, and Selection Suites

This appendix reports the controlled mathematical and synthetic simulation suites referenced in Sections I–K. None of these controlled benchmarks are claimed as live language-model or human-subject evidence.

### O.1 Synthetic Directed Acyclic Graph Interdiction

To isolate graph waist width $w$ while holding path multiplicity roughly constant (8.40M–7.56M source-to-sink pathways across 8-stage sparse DAGs, 200 graphs per family), Figure 8 compares Dinic minimum-cut interdiction [88] computed on Boltzmann-sampled red-team path reports against patching specific reported nodes. On narrow-waist graphs ($w = 6$), $N = 300$ path reports suffice for minimum-cut gating at budget $B = 6$ to block 94.6% of pathways and completely sever 73.5% of graphs (100.0% at $N = 3{,}000$), whereas patching $B = 32$ specific nodes severs 0.0%. On wide graphs ($w = 27$), even $N = 3{,}000$ reports sever only 6.0% at $B = 6$; on these synthetic graphs, report-driven severing needs a narrow waist. Across 2,048 small random DAGs, weighted vertex-splitting Dinic cuts match brute-force optimal subsets with 0 discrepancies.

### O.2 20D Six-Family Classifiers, Copula Quadrature, and Portfolio Ceilings

Table 17 and Figure 9 evaluate monitor failure correlation and multi-domain assessment ceilings:

- **Gaussian copula quadrature.** Using 200-node Golub–Welsch Gauss–Hermite quadrature [105] at single-monitor miss $m = 0.10$ (Figure 9a), equicorrelation $\rho = 0.6$ inflates 6-monitor joint miss $8,118\times$ over $m^6$ (0.81% vs. 0.0001%).
- **Six-family 20D classifier suite.** Across 24 synthetic teacher seeds (5% benign false-positive calibration across multilayer perceptron, random Fourier quadratic, stump ensemble, nearest-neighbor, radial-basis, and logistic classifiers; Table 17 and Figure 9b), architectural diversity on a *shared* step input still suffers $24.0\times$ passive ($\rho_{\text{adv}} = 0$) and $8.3\times$ adversarial ($\rho_{\text{adv}} = 2.0$) inflation at $k = 6$, whereas evaluating distinct step inputs tracks the independent product at the cost of 25.6% single-stage benign false blocks.
- **Multi-domain generalizability ceiling.** Under crossed random-effects generalizability theory [93] (10,000 subjects; Figure 9c), repeating a single benchmark 64 times stalls at $R^2_G = 18.9\%$ (single-domain ceiling 19.2%), whereas 64 diverse domains reach 73.7% below the shared-bias ceiling 83.3%. Similarly, scaling evaluator panel size to $M = 64$ on a single context $C_{\text{ctx}} = 1$ stalls at 23.4%, whereas

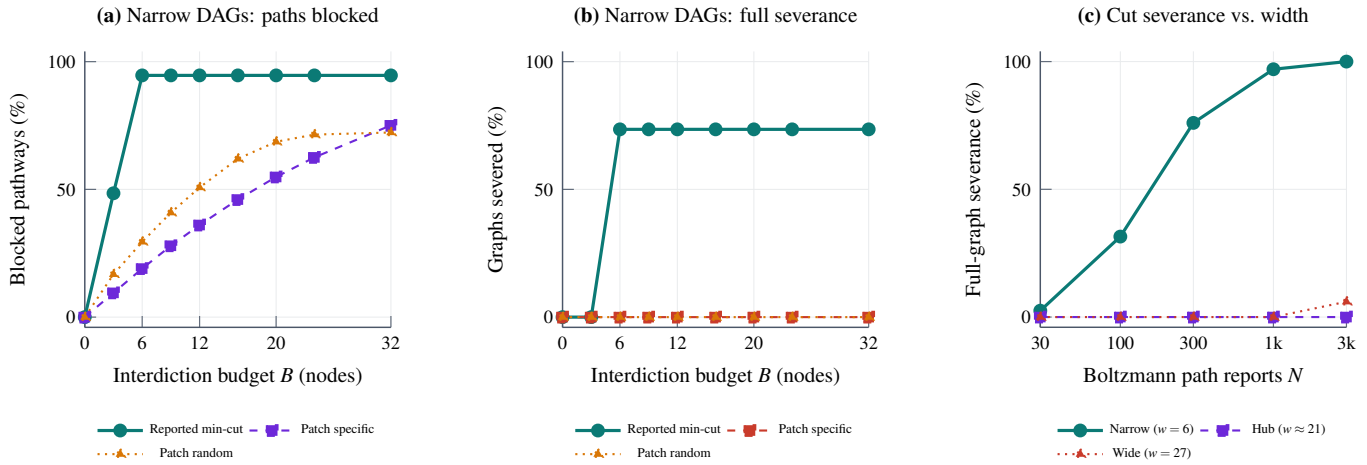



**Figure 8:** Synthetic 8-stage sparse directed acyclic graph interdiction (200 graphs per family, $\sim$ 7.6M–8.4M pathways). **(a,b)** On narrow-waist graphs ($w = 6$, $N = 300$ Boltzmann reports), reported-subgraph Dinic minimum cuts block 94.6% and completely sever 73.5% of graphs at $B = 6$, whereas route-specific patching severs 0% even at $B = 32$. **(c)** On wide graphs ($w = 27$), reported minimum cuts rarely sever the full graph even at $N = 3{,}000$, consistent with compact gating requiring a narrow bottleneck in these synthetic graphs.

| Synthetic 20D 6-gate architecture | $\rho_{adv}$ | Joint 6-miss | Indep. product | Ratio | Benign false block |
|---|---|---|---|---|---|
| Homogeneous MLP, shared step | 0.0 | $4.75 \pm 1.82\%$ | 0.0675% | 70.4× | 14.1% |
| 6-family heterogeneous, shared step | 0.0 | $5.21 \pm 1.55\%$ | 0.2171% | 24.0× | 16.9% |
| 6-family heterogeneous, distinct steps | 0.0 | $0.260 \pm 0.176\%$ | 0.2049% | 1.27× | 25.6% |
| Homogeneous MLP, shared step | 2.0 | $9.44 \pm 2.95\%$ | 0.47% | 19.9× | — |
| 6-family heterogeneous, shared step | 2.0 | $8.79 \pm 2.59\%$ | 1.06% | 8.3× | — |
| 6-family, shared + 8-query score feedback | 2.0 | $17.45 \pm 4.45\%$ | — | — | — |
| 6-family heterogeneous, distinct steps | 2.0 | $1.24 \pm 0.84\%$ | 0.98% | 1.26× | — |
| Shared-step read-only retry ($K = 32$ draws) | 2.0 | $72.4 \pm 10.4\%$ | 71.0% (law) | — | — |

**Table 17:** Controlled 20-dimensional synthetic monitor benchmark across 24 independent teacher seeds (multilayer perceptron, quadratic random Fourier features, axis stump ensemble, nearest neighbors, radial-basis prototype, and logistic linear classifiers; 5% per-gate benign false-positive calibration). Model diversity on a shared step leaves strong residual failure dependence, whereas independent step evidence tracks the marginal product at the cost of 25.6% benign false blocks. Controlled synthetic benchmark only, not frontier-model transfer evidence.

$M = 4$ across $C_{ctx} = 64$ contexts reaches 66.1%.

### O.3 Population Selection Co-Evolution

Table 18 reports 80-generation replicator dynamics ($n_{pop} = 2{,}000$, 16 replicates; finite-population Price covariance error $\leq 3.2e-14$). Ranking agents in top-10% tournaments selects high-variance gambling whether scored on raw output (95.1% gamble share, 4.54% harm) or conferred benefit (94.9% gamble share). Bottom-20% survival-floor elimination suppresses gambling to 3.8%—a 29.7× harm reduction (0.15%)—while retaining 64.5% of tournament capacity growth.

## P Frozen Holdout Accounting and Prospective Study Protocols

### P.1 Predeclared Holdout Design and Base-Unit Clustering

Before executing live model campaigns, we sealed $N = 32$ independent tool-use holdout tasks under SHA-256 cryptographic commitments alongside an evaluation plan specifying 16 family-wise primary comparisons ($\alpha = 0.05/16 = 0.003125$ per comparison) and a 5 pp usefulness non-inferiority margin. Under the predeclared marginal Clopper–Pearson union-bound confidence interval [106, 107] for paired binary differences, certifying a 5 pp margin at $\alpha = 0.05/16$ even with zero observed losses requires at least $N_{min} = 113$ independent cases. Because CPU execution budget capped the frozen pilot holdout at $N = 32$ cases per checkpoint, all predeclared marginal union-bound intervals remain explicitly inconclusive and are preserved without post-hoc replacement in Table 19.

Two methodological rules govern Table 19:

1. **Integration soundness vs. stochastic estimation.** Complete mediation shares its formal capability specification with the safety grader on cooperative sinks. Observing 0 mediated unauthorized commits across all holdout and AgentDojo campaigns is therefore a regression check that the broker and sink enforce that specification on these inputs. It does not establish soundness, and it is not an empirical probability bound on unmodeled hazards.
2. **Strict clustering by independent base unit.** Running two checkpoints (Qwen3-4B and Qwen3-8B) on the same 32 holdout cases or evaluating multiple injection/withdrawal variants on the same 12 AgentDojo tasks induces within-task correlation. Pooled rows in Table 19 therefore cluster strictly at the independent base unit ($N = 32$ holdout cases, where $14/32$ cases exhibit unmediated breach on at least one checkpoint vs. $0/32$ mediated; and $N = 12$ AgentDojo

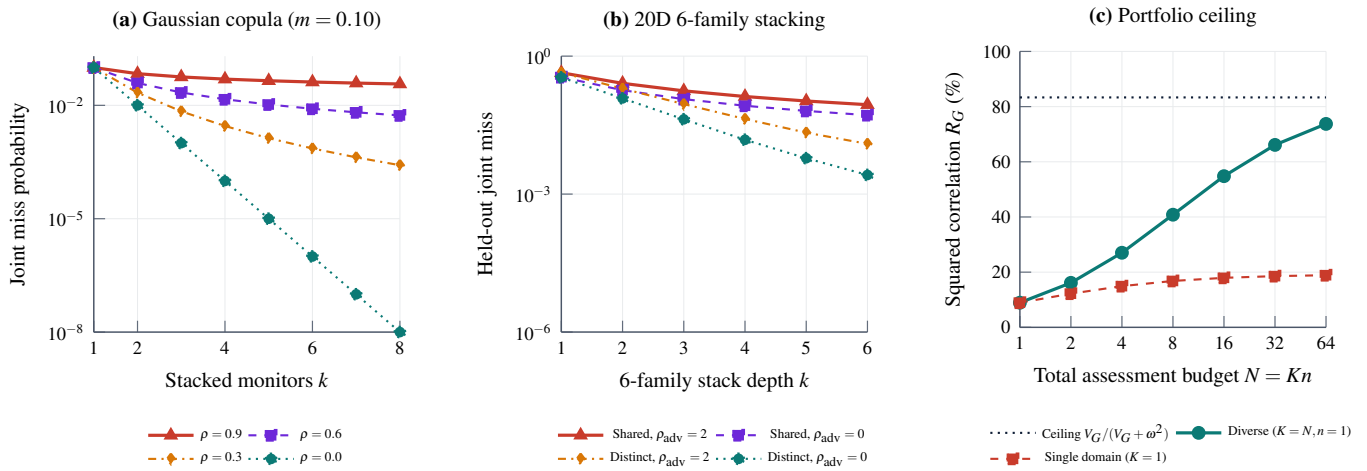



**Figure 9:** Correlated monitor stacking and multi-signal assessment ceilings. **(a)** Exact 200-node Golub–Welsch Gaussian copula miss rates: at $\rho = 0.6, k = 6$, joint miss is $8,118\times$ $m^6$. **(b)** Six distinct classifier families on a shared 20-dimensional input still plateau orders of magnitude above distinct-step gates. **(c)** Repeating one benchmark ($K = 1, n = 64$) stalls at 18.9%, while 64 diverse domains reach 73.7% below the shared-bias ceiling (83.3%).

| Selection regime ($N = 2{,}000,\ G = 80,\ 16$ replicates) | Final gamble share | Capacity gain $\Delta c$ | Harm rate | Population welfare |
|---|---|---|---|---|
| Taken-output top-10% contest (warrior) | $95.1 \pm 0.3\%$ | $+5.58 \pm 0.12$ | 4.54% | 3.30 |
| Conferred-benefit top-10% contest | $94.9 \pm 0.4\%$ | $+5.45 \pm 0.16$ | 4.76% | 3.22 |
| Concave capped + harm-penalized score | $9.9 \pm 0.2\%$ | $+1.55 \pm 0.09$ | 0.72% | 1.45 |
| Bottom-20% harm/floor elimination + delivered growth | $3.8 \pm 0.1\%$ | $+3.60 \pm 0.19$ | 0.15% | 2.91 |
| Caretaker higher-cap variant (cap $= 4$, pen $= 6$) | $7.5 \pm 0.4\%$ | $+4.53 \pm 0.19$ | 0.31% | 3.79 |

**Table 18:** Co-evolution of (gamble share $s$, capacity $c$) under stipulated payoff rules. Winner-take-most contests select reckless gambling even when ranked on conferred benefit; floor elimination suppresses gambling (3.8%, a $29.7\times$ harm reduction vs. warrior) while retaining 64.5% of warrior capacity growth. Mathematical payoff mechanics only, not RLHF or human-participant data.

base tasks, where $10/12$ tasks breach unprotected vs. $0/12$ mediated).

### P.2 Earlier Predictions and the Human Grant-Reversal Protocol

The previous version listed three further predictions. P5, that operators confer wider scopes on a deferential agent with lower fixed-scope error and so incur more total harm, is superseded by the conferral design in Appendix E. P6 extrapolated from synthetic simulations to the post-training of frontier models; it is withdrawn, because nothing in this paper bears on it. P7, that a larger frozen holdout would clear the noninferiority bound, was tested by E1 (Section 7).

**Human delegation protocol (for P5).** The previous version's two-arm protocol is superseded by the design in Appendix E, which controls competence and caution separately and measures harm at a mediated sink; only that design is predeclared for this version.

## Q Verification Boundaries and Reproducibility Summary

### Q.1 Trusted Computing Base and Verification Scope

Table 1 (§2) and Table 2 (§3) distinguish formally verified obligations from empirically tested system components. The Verus proofs establish the six properties listed in Section 3 for the verified kernel functions: conservation and post-fence exclusion, the canonical permit encoding, $d$-hop capability attenuation, relational exclusion, the monotone policy ratchet, and skew-buffered quorum quiescence. They do not establish full functional correctness of the broker and sink, whose decision logic, RPC handling and storage are unverified (249 verified obligations; 10/10 soundness mutations rejected).

Components outside the Verus specification belong to the empirical trusted computing base: the Linux kernel enforcing namespaces, Landlock v4, and seccomp-BPF; the C SQLite engine executing write-ahead-logged transactions (`FULL` synchronous durability for the broker and sink of the experiments, `NORMAL` for the three-process cluster test); physical monotonic clock drift bounded by Assumption 1 ($|t_{\text{shard}} - t_{\text{broker}}| \leq \delta_{\max}$); out-of-band human custody of Ed25519 steward root keys; and any proxy that wraps an external API to enforce a fence for it (Section 8). A vulnerability or misconfiguration in any of those lower layers can bypass kernel-level invariants without invalidating the SMT theorems.

### Q.2 Deterministic Artifact Reproduction

Every number, table and figure in this PDF is generated from committed reference artefacts without manual number entry, and the build checks that the generated files match those artefacts byte for byte. The validators recompute each reported number from raw artefacts; they are replay checks that share code with the harnesses, not independent implementations (Section 9).

| Paired comparison (clustered by independent unit) | $(n_{10}, n_{01})/N$ | $\hat{\delta}$ | Union $U_{1-\alpha}$ | Preregistered / clustered accounting status |
|---|---|---|---|---|
| Qwen3-4B holdout vs. Unmediated | $(0,10)/32$ | $-31.2$ pp | $+7.4$ pp | Predeclared ($N = 32$ cases); marginal union bound inconclusive ($N = 32 < 113$) |
| Qwen3-4B holdout vs. Same-model review | $(0,7)/32$ | $-21.9$ pp | $+12.8$ pp | Predeclared ($N = 32$ cases); marginal union bound inconclusive ($N = 32 < 113$) |
| Qwen3-8B holdout vs. Unmediated | $(0,4)/32$ | $-12.5$ pp | $+16.7$ pp | Predeclared ($N = 32$ cases); marginal union bound inconclusive ($N = 32 < 113$) |
| Qwen3-8B holdout vs. Same-model review | $(0,2)/32$ | $-6.2$ pp | $+18.1$ pp | Predeclared ($N = 32$ cases); marginal union bound inconclusive ($N = 32 < 113$) |
| Case-clustered holdout vs. Unmediated | $(0,14)/32$ | $-43.8$ pp | — | Clustered by base case ($N = 32$); descriptive discordance (0 mediated breaches by gate soundness) |
| Case-clustered holdout vs. Same-model review | $(0,9)/32$ | $-28.1$ pp | — | Clustered by base case ($N = 32$); descriptive discordance (0 mediated breaches by gate soundness) |
| Task-clustered AgentDojo vs. Unprotected | $(0,10)/12$ | $-83.3$ pp | — | Clustered by base task ($N = 12$); descriptive discordance (0 mediated breaches by gate soundness) |
| Task-clustered AgentDojo vs. Progent | $(0,4)/12$ | $-33.3$ pp | — | Clustered by base task ($N = 12$); descriptive discordance (0 mediated breaches by gate soundness) |

**Table 19:** The previous version's clustered ledger, kept for the record and superseded by E1 (Section 7). Rows marked predeclared are the pilot's planned comparisons; the clustering and every pooled row were added after the data. Its holdout rows pool both checkpoints, which E1's plan does not allow, and its Progent row counts budget overruns, a property Progent does not claim to enforce. Complete mediation's zero unauthorized commits hold by construction. The predeclared union-bound intervals were inconclusive at the pilot's size.